

- EpiProfile_PLANTS — Manual (Draft)
 - 1. What this repository is (and is not)
 - 2. The pipeline in one page
 - 3. Directory conventions
 - 4. Step A — Inventory and select ProteomeXchange files (public PXDs)
 - 5. Step B — Download RAW/WIFF from PX
 - 6. Step C/D — Convert to mzML, then extract MS1/MS2
 - 7. What “done” looks like at the end of the pre-processing block
 - 8. Next sections to write (placeholders)
 - 9. Change log for this manual
 - 10. Step E — Run the MATLAB core (EpiProfile_PLANTS)
 - 10.1 What you need before MATLAB
 - 10.2 How the runner organizes outputs
 - 10.3 Minimal MATLAB “smoke test” (1–2 runs)
 - 10.4 What files to look at first after a run
 - A) Folder creation (per run)
 - B) Per-run “detail” artifacts
 - C) Cohort-level tables
 - 10.5 Common pitfalls (and how to avoid them)
 - raw_names mismatch
 - Reusing a folder with an old RT reference
 - Too strict mass tolerance
 - Partial runs and mixed outputs
 - 11. Step F — First-pass QC (quick)
 - 12. Next to document
 - 13. Input file naming conventions (MS1/MS2)
 - 13.1 The key rule: one run = one basename
 - 13.2 File pairing requirement
 - 13.3 Recommended naming convention
 - 13.4 Allowed characters (practical rule)
 - 13.5 Where the files should live
 - 13.6 Quick sanity check before running MATLAB
 - 13.7 Building raw_names from filenames (MATLAB helper)
 - 13.8 If your extractor produces different names
 - 14. Retention-time (RT) reference system
 - 14.1 What the RT reference is
 - 14.2 Where the RT reference is stored
 - 14.3 When it is created and when it is reused
 - 14.4 The main failure mode: cross-dataset contamination
 - 14.5 Safe folder hygiene rules (recommended)
 - 14.6 Conservative mode (no RT reference)
 - 15. Output contracts (what files mean)
 - 15.1 Per-run folder structure
 - 15.2 Cohort-level table: histone_ratios.xls (important)
 - 15.2.1 Typical structure (two header rows)
 - 15.3 Output contract: long-format table (recommended downstream)
 - 16. Parsing histone_ratios.xls safely (R and Python)
 - 16.1 R (readr + tidyr) — conservative template
 - 16.2 Python (pandas) — conservative template
 - 17. Troubleshooting by symptom (quick)

- 17.1 “Everything is missing / outputs are mostly empty”
 - 17.2 “Some runs look fine, one run is almost blank”
 - 17.3 “RT looks wrong (shifted)”
- 18. Reproducibility checklist (minimum)
- 19. Next section to write
- 20. RT reference mechanism (detailed)
 - 20.1 Definitions
 - 20.2 What “global by folder” means
 - 20.3 What gets cached
 - 20.4 When the cache helps (intended use)
 - 20.5 When the cache hurts (failure cases)
 - 20.6 Operational modes (safe patterns)
 - Mode A — Stable batch mode (reuse allowed)
 - Mode B — Conservative mode (no reuse)
 - Mode C — Split-by-method mode (recommended default)
 - 20.7 Minimal documentation to record
- 21. Output schema (long-format contract)
 - 21.1 Required columns
 - 21.2 Recommended derived columns (strongly suggested)
 - 21.3 Allowed values and invariants
 - Metric invariants
 - Feature uniqueness
 - 21.4 Validation checks (minimum)
- 22. Minimal QC outputs (what to keep)
- 23. Run manifest template (copy/paste)
- 24. Next section to write
- 25. Feature parsing and mapping rules
 - 25.1 Why you need parsing
 - 25.2 Minimal parsing outputs
 - 25.3 Conservative region extraction
 - 25.4 PTM token extraction (basic)
 - 25.5 Mark aggregation using include/exclude patterns
 - 25.6 Validator: catch inconsistent assignments
- 26. Anchor peptides for QC
 - 26.1 What makes a good anchor
 - 26.2 How to use anchors
- 27. Minimal end-to-end walkthrough (one run)
 - 27.1 Prepare one run
 - 27.2 Run MATLAB
 - 27.3 Verify outputs
 - 27.4 Parse the cohort export (text-first)
- 28. File mapping (samples → conditions)
- 29. Next section to write
- 30. Core data model (hDP, hPF, hPTM)
 - 30.1 hDP — histone-derived peptide (peptide level)
 - 30.2 hPF — histone peptideform (modified form level)
 - 30.3 hPTM — mark summary (marginal PTM level)
 - 30.4 Where ratios live (composition semantics)
- 31. Default mark list (Arabidopsis-oriented)
 - 31.1 H3 / H3.3 marks (typical)
 - 31.2 H4 marks (typical)

- 31.3 Example include/exclude rules (pattern-based)
- 32. Filtering strategy (minimum recommended)
 - 32.1 Feature-level filters (hPF level)
 - 32.2 Peptide-level filters (hDP level)
 - 32.3 Mark-level filters (hPTM level)
- 33. QC decision tree (minimal)
 - 33.1 If missingness is high in many samples
 - 33.2 If one sample is an outlier in intensity
 - 33.3 If RT drift is systematic
 - 33.4 If ratios look “flat” (no dynamics anywhere)
- 34. Next section to write
- 35. Worked example: from histone_ratios to hDP/hPTM + minimal QC
 - 35.1 Inputs
 - 35.2 R worked example (readr/tidyr)
 - 35.2.1 Parse TSV-disguised export to long format
 - 35.2.2 Add structured parsing: Region, Histone, PTM tokens
 - 35.2.3 Compute hDP-level totals and (re)derive compositional ratios if needed
 - 35.2.4 Define mark rules and compute hPTM marginal sums
 - 35.2.5 Minimal QC summaries (counts, intensity scale, missingness)
 - 35.3 Python worked example (pandas)
 - 35.3.1 Parse TSV-disguised export to long format
 - 35.3.2 Parse Region/Histone/PTM tokens
 - 35.3.3 Pivot to wide and compute Ratio from Area
 - 35.3.4 Mark rules + validator + marginal sums
 - 35.3.5 Minimal QC summaries
- 36. Packaging recommendations (keep the workflow auditable)
- 37. Next section to write
- 38. Canonical Feature string specification (practical contract)
 - 38.1 Minimal required elements
 - 38.2 Allowed PTM token vocabulary (recommended)
 - 38.3 Separator conventions (do not rely on them)
 - 38.4 Recommended policy for new layouts
- 39. Invariant checks (fail fast)
 - 39.1 Parsing invariants (table-level)
 - 39.2 Composition invariants (hDP-level)
 - 39.3 Mark-rule invariants (hPTM validator)
- 40. Minimal tests (R and Python)
 - 40.1 R (base + stopifnot) — minimal checks
- 45. Expected folder tree (reference example)
 - 45.1 Dataset root (public PXD example)
 - 45.2 MATLAB output folder (created under raw_path)
 - 45.3 “Green flags” (quick signs of a healthy run)
 - 45.4 “Red flags” (stop and fix before analysis)
- 46. QC packet template (what to save)
 - 46.1 Minimum QC tables
 - 46.2 Minimum QC plots
 - 46.3 Naming rules
- 47. Glossary (short)
- 48. Quickstart checklist (copy/paste)
- 49. Next section to write
- 50. ProteomeXchange setup (R): 00_complete_pipeline_setup.R

- 50.1 What it does (as written)
 - 50.2 Important notes (based on the file content)
- 51. Downloader (R): 01_download_raw.R
 - 51.1 What it does (as written)
 - 51.2 Notes you should not miss
- 52. Shiny converter (R): EpiProfile_PLANTS – Shiny Converter.R
 - 52.1 Dependencies (as declared)
 - 52.2 Expected external scripts (hard-coded paths)
 - 52.3 WIFF → mzML: how it runs it
 - 52.4 mzML → MS1/MS2: how it runs it
- 53. mzML QC helper (bash): QC_MZML_AUTO.sh
- 54. Interactive analysis prototype (R/Shiny): Análisis de EpiProfile - Histonas Arabidopsis.R
 - 54.1 What is clearly implemented
 - 54.2 What is not implemented (in this file)
- 55. Next section (grounded): “How these scripts map to the repository pipeline”
- 55. How these scripts map to the repository pipeline
 - 55.1 Script → role (what it is for)
 - 55.2 The “two roots” you will see in practice
 - 55.3 Verified per-PXD directory layout (created by 00_complete_pipeline_setup.R)
 - 55.4 Download step (where files really land)
 - 55.5 Conversion + extraction step (what the Shiny Converter expects)
 - 55.6 Where QC_MZML_AUTO.sh fits (and what it really does)
 - 55.7 Hand-off point to MATLAB (pipeline intent)
 - 55.8 Minimal mental model (one screen)
 - 55.9 What to document next (still grounded)
- 56. Workspace alignment: PX_DATA scaffold vs Shiny Converter expectations
 - 56.1 What the Shiny Converter assumes (as written)
 - 56.2 What the PX_DATA scaffold creates (as written)
 - 56.3 Two consistent ways to run (pick one per dataset)
- 57. Where to put the helper scripts (converter requirements)
- 58. Logging behavior (what you actually get today)
 - 58.1 Downloader (01_download_raw.R)
 - 58.2 Shiny Converter (EpiProfile_PLANTS – Shiny Converter.R)
 - 58.3 mzML QC wrapper (QC_MZML_AUTO.sh)
- 59. Pipeline hand-off: what MATLAB expects from pre-steps
- 60. What we can document next without guessing
- 61. Deployment notes (Windows + WSL) — what this repo actually runs
 - 61.1 Minimum runtime assumptions (because the code calls them)
 - 61.2 “Project root” rule (do not miss this)
- 62. Running the Shiny Converter (verified behavior)
 - 62.1 What you must provide (inputs in UI)
 - 62.2 What the app will do (as coded)
 - 62.3 Where logs go
- 63. Running the ProteomeXchange staging scripts (verified behavior)
 - 63.1 00_complete_pipeline_setup.R (inventory + scaffold)
 - 63.2 01_download_raw.R (download RAW/WIFF)
- 64. Bridging the two worlds: PX_DATA scaffold vs Shiny Converter
 - 64.1 Why they don’t align by default
 - 64.2 The conservative bridge (no code changes)
- 65. Output export formats: detect first, parse second
 - 65.1 Pattern A — “classic EpiProfile” row layout (what the analysis Shiny app assumes)

- 65.2 Pattern B — “2-row header table” (what the conservative pandas template assumes)
- 65.3 Detection rule (simple)
- 66. Known limitations of the existing Shiny analysis parser (do not rely on it blindly)
- 67. Canonical sequence notes (internal project files)
- 68. Next sections to write (still “no invention”)
- 69. Minimal reproducible run recipes (what you can do today)
 - 69.1 Recipe A — ProteomeXchange staging (inventory → download)
 - 69.2 Recipe B — Local conversion (WIFF → mzML → MS1/MS2) via Shiny
- 70. “What to edit first” in each script (verified knobs)
 - 70.1 00_complete_pipeline_setup.R
 - 70.2 01_download_raw.R
 - 70.3 EpiProfile_PLANTS – Shiny Converter.R
- 71. Known “footguns” in this repo snapshot (verified from code)
 - 71.1 00_complete_pipeline_setup.R: inconsistent download block
 - 71.2 01_download_raw.R: self-sourcing tail
 - 71.3 Análisis de EpiProfile - Histonas Arabidopsis.R: not a canonical parser
- 72. Verified interface contracts between the Shiny Converter and external scripts
 - 72.1 01_convert_wiff_to_mzML.sh
 - 72.2 02_extract_MS1_MS2.ps1
- 73. Confirming “ready for MATLAB” state (before you open MATLAB)
- 74. Next sections to write (still grounded, but requires repo-wide search)
- 75. Canonical peptide motifs (verified) for Arabidopsis / Marchantia / Chlamydomonas
 - 75.1 H3 N-terminus: TKQTAR and the K9/K14 peptide (verified positions)
 - 75.2 H3 K27–K36 block: species/isoform signature motifs (verified)
 - 75.3 H4 motifs: what is conserved, and what changes (verified)
- 76. Layout inventory table (grounded in the repo’s audit notes)
 - 76.1 H3 panels (from notas_mapping_multi_sp)
 - 76.2 H4 panels (from notas_README_canonical)
- 77. How to use the motif tables to decide “layout portability” (minimal rules)
- 78. Suggested next section to write (consistent with this manual’s flow)
- 79. Panel cards (layouts): a “one page per peptide” index you can trust
 - 79.1 Panel card template (canonical fields)
 - 79.2 H3 panel inventory (verified from notas_mapping_multi_sp)
 - 79.3 H4 panel inventory (verified from notas_README_canonical)
 - 79.4 Panel cards (H3) — verified summaries, no extra assumptions
 - 79.4.1 H3_01_3_8 (H3K4 panel)
 - 79.4.2 H3_02_9_17 (K9/K14 panel; 10-state core)
 - 79.4.3 H3_02a_9_17 (S10ph-centric twin of the K9/K14 panel)
 - 79.4.4 H3_02b_9_17 (S10ac / S10pr panel)
 - 79.4.5 H3_03_18_26 (K18/T22/K23 panel)
 - 79.4.6 H3_04a_27_40 (S28-centered child panel using inherited RTs)
 - 79.4.7 H3_06a_53_63 (derivatization regioisomer monitor)
 - 79.4.8 H3_07_73_83 (H3K79 core panel)
 - 79.4.9 H3_08_117_128 (H3 C-terminal panel: M120ox + K122ac)
 - 79.5 Panel cards (H4) — verified summaries, no extra assumptions
 - 79.5.1 H4_01_4_17 (H4 N-terminus Kac combinatorics)
 - 79.5.2 H4_02_20_23 (H4K20 short peptide panel)
 - 79.5.3 H4_02c_20_36 (H4K20 Arg-extended auxiliary panel)
 - 79.5.4 H4_04_40_45 (H4K44ac panel)
 - 79.5.5 H4_05_68_78 (Y72 O-propionylation artifact monitor)
- 80. Suggested next section (what to write next, and why)

- 81. What a “layout panel” produces (verified output contract)
- 82. What to inspect first inside a panel .mat (verified fields)
- 83. How RT anchoring + drift correction works (worked example: H3_01_3_8, verified)
 - 83.1 Shapes of the core matrices (verified)
 - 83.2 Anchor on unmod (verified)
 - 83.3 Extract unmod with `get_histone0`, then apply drift correction (verified)
 - 83.4 Relocate each PTM row, then extract with `get_histone1` (verified)
 - 83.5 Why `find_pair` exists (verified purpose)
 - 83.6 The “timeline” idea (verified for H3_01_3_8)
- 84. `mod_type` dialect (verified minimal grammar)
- 85. Peptide inventory tables for portability (verified from repo bundles)
 - 85.1 H4 peptide inventory (as documented in `notas_README_canonical`)
 - 85.2 H3 peptide inventory (extracted from H3_AT_MP_CR.txt bundle)
- 86. “Core” vs “signature” peptides (verified concept from `notas_README_canonical`)
- 87. Minimal “panel portability” checklist (sequence + layout-level)
- 88. Suggested next section (still grounded): “From layouts to a validated cohort export”
- 88. Suggested next section (still grounded): “From layouts to a validated cohort export”
 - Locate the cohort export writer (mechanical)
 - Fast search (recommended: `ripgrep`)
 - If the repo has multiple MATLAB roots
 - How to decide “this is the writer”
 - Confirm it’s the combined export (not per-run detail)
 - Cohort export contract (documented from the writer)
 - File discovery (what the pipeline guarantees)
 - 57.2 File type: “.xls” is often TSV (text)
 - 57.3 Header schema (two-row header)
 - 57.4 Body schema (rows)
 - 57.5 Metrics (columns)
 - 57.6 Minimal parsing template (Python)
- 58. Export validation (must be part of the contract)
 - 58.1 Structural checks
 - 58.2 Ratio invariants
 - 58.3 Quick validator skeleton (Python)
- 59. What you must extract from the MATLAB writer (for line-by-line documentation)

EpiProfile_PLANTS — Manual (Draft)

This document explains how to run EpiProfile_PLANTS on plant histone PTM data, from raw files to the main quantitative outputs. It is written for new users and keeps assumptions explicit.

Status: work in progress. The first goal is “you can run it once end-to-end without surprises”.

1. What this repository is (and is not)

EpiProfile_PLANTS is a plant-adapted implementation of an EpiProfile-style workflow for histone PTM quantification from LC–MS/MS data, using curated peptide layouts and retention-time-guided extraction.

It expects that, at some point, your vendor raw data becomes mzML and then becomes MS1/MS2 text-like inputs for the MATLAB core. This repo includes helper scripts for those “pre-EpiProfile” steps (inventory,

download, conversion, QC), but those steps are modular: you can swap them out if your lab already has a stable conversion path.

2. The pipeline in one page

- A. Select datasets (public PXDs or your own runs)
- B. Download RAW/WIFF
- C. Convert to mzML
- D. Extract MS1/MS2 from mzML
- E. Run EpiProfile_PLANTS (MATLAB)
- F. Inspect outputs + QC plots; move to downstream R analysis

The repo provides scripts for A–D and basic visualization/QC helpers. For public PXDs, the inventory + download path is already scripted. :contentReference[oaicite:0]{index=0} :contentReference[oaicite:1]{index=1}

3. Directory conventions

A consistent folder layout avoids most user errors. The scripts assume a root directory with two main areas:

- **PX_INVENTORY/**
Tables describing what exists in each PXD and which files are selected.
- **PX_DATA/**
One folder per PXD, with standardized subfolders for raw, mzML, MS1_MS2, outputs, logs.

The setup script creates this structure automatically. :contentReference[oaicite:2]{index=2}

Suggested layout (simplified):

- **EpiProfile_2.0_PLANTS/**
 - **PX_INVENTORY/**
 - **PX_DATA/**
 - **PXDxxxxxx/**
 - **raw/**
 - **wiff/**
 - **raw_thermo/**
 - **mzML/**
 - **MS1_MS2/**
 - **epiprofile_output/**
 - **metadata/**
 - **logs/**

4. Step A — Inventory and select ProteomeXchange files (public PXDs)

Goal: create a local inventory of all files in each PXD, then mark which RAW/WIFF files you want to process.

Script: `00_complete_pipeline_setup.R` :contentReference[oaicite:3]{index=3}

What it does (conceptually):

1. Connects to each PXD ID you list.
2. Pulls the file table.
3. Classifies files by extension (RAW, WIFF/WIFF.SCAN, mzML, ID files).
4. Writes:
 - `px_files_inventory_<timestamp>.tsv`
 - `px_files_summary_<timestamp>.tsv`
 - `px_projects_<timestamp>.tsv`
 - `px_files_selected_<timestamp>.tsv` (this is your “download list”)

Minimal usage:

- Open the script.
- Edit `pxd_vec` to the set of PXDs you want.
- Set `ROOT_DIR` to your local working root.
- Run the script in R.

Expected output:

- A new `PX_INVENTORY/px_files_selected_<timestamp>.tsv` that feeds the downloader.

5. Step B — Download RAW/WIFF from PX

Goal: download only the selected RAW/WIFF files into the standardized `PX_DATA/<PXD>/raw/...` locations.

Script: `01_download_raw.R` :contentReference[oaicite:4]{index=4}

Key behavior:

- Automatically finds the most recent `px_files_selected_*.tsv`.
- Downloads each file with retries and an HTTPS fallback.
- Writes incremental status files so you can resume without guessing.

What you should check before running:

- `ROOT_DIR` points to the same root used in Step A.
- Your disk has enough space (PXD files can be large).
- You can access PRIDE/ProteomeXchange URLs from your network.

6. Step C/D — Convert to mzML, then extract MS1/MS2

These two steps are often the most environment-specific part of the workflow.

C) WIFF/WIFF.SCAN → mzML (typically via ProteoWizard `msconvert`)

D) mzML → MS1/MS2 (extract-xml / pXtract-style extraction)

This repo includes a Shiny “converter” wrapper that calls external tools (WSL + Docker for msconvert; PowerShell for extraction). Use it if you want a guided UI instead of running scripts manually.

:contentReference[oaicite:5]{index=5}

Notes:

- The converter does not replace msconvert/extractors. It orchestrates them.
- If you already produce mzML and MS1/MS2 elsewhere, you can skip the UI and just place outputs into the expected folders.

Optional: mzML QC automation script (resource-aware mode selection). :contentReference[oaicite:6]{index=6}

7. What “done” looks like at the end of the pre-processing block

Before you run MATLAB, you should be able to point to:

- A folder containing mzML files (one per run), and/or
- A folder containing MS1/MS2 extracted files (one pair per run)

If you are processing multiple runs, keep them grouped by “homogeneous acquisition conditions” (same LC method / instrument / general RT scale). Mixing heterogeneous runs in one shared working folder is a common source of retention-time reference problems later.

(Details on RT reference mechanics and safe folder hygiene will be documented in a later section.)

8. Next sections to write (placeholders)

- Step E — Running the MATLAB core (inputs, parameters, expected outputs)
- Output contracts (what files mean; what is “histone_ratios” and how to parse it)
- Minimal QC checklist (what plots/tables to inspect every time)
- Troubleshooting by symptom (RT=0, missing peptides, empty windows, layout mismatch)
- Reproducibility checklist (pin versions, folder hygiene, run manifests)

9. Change log for this manual

- Draft created: 2026-02-12

10. Step E — Run the MATLAB core (EpiProfile_PLANTS)

This step reads the extracted MS1/MS2 files, matches them to the histone layouts, integrates XIC areas, and writes the standard EpiProfile outputs.

10.1 What you need before MATLAB

You should already have, for each LC–MS/MS run:

- MS1 extraction file(s)
- MS2 extraction file(s)
- A consistent basename per run (this becomes `raw_name`)

All runs you process together should have comparable LC conditions (same gradient/column family and similar RT scale). Do not mix “very different” acquisitions in one working folder.

10.2 How the runner organizes outputs

The main runner creates an output root:

- `<raw_path>/histone_layouts/`

Inside it, each run gets a numbered folder based on its position in `raw_names`:

- `01_<raw_name>/`
- `02_<raw_name>/`
- ...
- Each run also gets a `detail/` subfolder for per-peptide plots and intermediate exports.

This means the order of `raw_names` matters. Keep it stable if you want reproducible folder naming.

10.3 Minimal MATLAB “smoke test” (1–2 runs)

In MATLAB:

1. Add the EpiProfile_PLANTS MATLAB code to the path.
2. Define `raw_path`, `raw_names`, and a mass tolerance `ptol`.
3. Run the driver function.

Example:

```
% 1) Point MATLAB to the EpiProfile_PLANTS MATLAB code
addpath(genpath('/path/to/EpiProfile_PLANTS/matlab'));

% 2) Define the dataset folder (contains MS1/MS2 extracted files)
raw_path = '/path/to/PX_DATA/PXDxxxxxx/MS1_MS2';

% 3) Basenames of runs (must match your extracted file naming)
raw_names = {'Run_001', 'Run_002'};

% 4) Mass tolerance (units depend on your build; commonly ppm)
ptol = 10;

% 5) Special mode flag (keep 0 unless you know you need it)
special = 0;

% 6) Run
DrawISOProfile1(raw_path, raw_names, ptol, special);
```

If your naming is correct, you should see the `histone_layouts/` folder appear and populate during the run.

10.4 What files to look at first after a run

Start with these checks.

A) Folder creation (per run)

- `<raw_path>/histone_layouts/01_<raw_name>/`
- `<raw_path>/histone_layouts/01_<raw_name>/detail/`

B) Per-run “detail” artifacts

(Content varies by build, but typically includes:)

- Peptide-level XIC plots
- Identification lists / PSM summaries
- Intermediate tables used to build cohort matrices

C) Cohort-level tables

- A combined export containing ratios (and often areas and RT blocks), used for downstream R analysis.

Important: in this project, files named `*.xls` may actually be tab-separated text (TSV). Treat them as text unless you have verified they are real Excel files.

10.5 Common pitfalls (and how to avoid them)

`raw_names` mismatch

If the basename in `raw_names` does not match the extracted file naming, the run will fail early or produce empty outputs. Fix by renaming files or adjusting `raw_names`.

Reusing a folder with an old RT reference

EpiProfile-style workflows often store retention-time reference artifacts under `histone_layouts/` (for example a `0_ref_info.mat`). If you reuse the same `raw_path` for a different dataset/instrument, that reference can be applied unintentionally and cause wrong RT-guidance (or failures).

Rule of thumb: one homogeneous dataset per `raw_path`. If you repurpose a folder, delete the old `histone_layouts/` first.

Too strict mass tolerance

If `ptol` is too strict, features may not be matched and outputs will look “empty”. If you are unsure, start slightly looser and tighten later.

Partial runs and mixed outputs

If MATLAB crashes mid-run, you can end up with half-built folders. Clean the affected `01_<raw_name>` folder before re-running, or you may read stale intermediate files.

11. Step F — First-pass QC (quick)

Do this once per dataset, before any statistics:

- Count: do you see the expected number of histone peptides/features reported (roughly, not exactly)?
- Intensity distribution: do per-run areas look comparable, or is one run clearly off-scale?
- Missingness: are there runs with near-zero detections across many peptides?
- RT sanity: do the reported RTs for major peptides look consistent across runs?

If any of these checks fail, stop and fix the input set (conversion, extraction, naming, folder hygiene) before doing downstream normalization.

12. Next to document

13. Input file naming conventions (MS1/MS2)

This workflow is strict about one thing: each LC–MS/MS run must map to a single `raw_name`. That `raw_name` is used in MATLAB to (i) find the right MS1/MS2 inputs and (ii) label outputs under `histone_layouts/`.

13.1 The key rule: one run = one basename

Pick a basename that identifies the run (no extension). Examples:

- `Run_001`
- `Aging_YNG_rep1`
- `PXD046034_2023-10-12_01`

Then make sure both the MS1 and MS2 extracted files for that run contain that basename in their filename.

In MATLAB you will pass:

- `raw_path` = folder that contains the extracted MS1/MS2 files
- `raw_names` = a list of basenames (one per run)

13.2 File pairing requirement

For each entry `raw_name` in `raw_names`, you must have:

- exactly one MS1 file for that run
- exactly one MS2 file for that run

If you have multiple MS1/MS2 files that match the same basename, you will get ambiguity (best case: wrong pairing; worst case: empty outputs).

13.3 Recommended naming convention

Use one of these simple patterns and keep it consistent across all runs:

Option A (explicit suffix, easiest to read)

- `<raw_name>_MS1.txt`
- `<raw_name>_MS2.txt`

Option B (short extensions)

- `<raw_name>.ms1`
- `<raw_name>.ms2`

Option C (extractor-style text)

- `<raw_name>_MS1.tsv`
- `<raw_name>_MS2.tsv`

Whatever you choose, keep the basename identical between the MS1 and MS2 pair.

13.4 Allowed characters (practical rule)

To avoid issues across Windows/WSL/Linux and MATLAB string handling:

- Use letters, numbers, `_` and `-`
- Avoid spaces
- Avoid parentheses, brackets, `#`, `&`, `;`
- Avoid very long names (keep < 80 characters)

13.5 Where the files should live

Keep all MS1/MS2 extracted files for a dataset in a single folder and point `raw_path` to it, for example:

- `.../PX_DATA/PXDxxxxxx/MS1_MS2/`

Do not split MS1 and MS2 into different folders unless your MATLAB code is explicitly written to do that.

13.6 Quick sanity check before running MATLAB

From a terminal:

```
ls -1 /path/to/MS1_MS2 | head
```

You should be able to visually confirm that for each run you have one MS1-like file and one MS2-like file sharing the same basename.

13.7 Building `raw_names` from filenames (MATLAB helper)

If your MS1/MS2 folder is clean and uses a consistent suffix, you can derive `raw_names` automatically.

Example for Option A:

```

raw_path = '/path/to/MS1_MS2';

ms1_files = dir(fullfile(raw_path, '*_MS1.*'));
raw_names = erase({ms1_files.name}, '_MS1.txt'); % adjust extension if needed

% Optional: verify each raw_name has a matching MS2 file
for i = 1:numel(raw_names)
    ms2_match = dir(fullfile(raw_path, [raw_names{i}, '_MS2.*']));
    if isempty(ms2_match)
        fprintf('Missing MS2 for: %s\n', raw_names{i});
    end
end

```

Adjust the erase() string to match your real suffix/extension.

13.8 If your extractor produces different names

Some extractors include extra tokens (scan range, instrument, timestamps). In that case:

Decide what the stable “run ID” is (the part you want as raw_name).

Rename files to enforce one clean basename per run, or write a short mapping layer to translate filenames → raw_names.

Do not try to “guess” basenames on the fly during analysis. It makes runs non-reproducible and breaks downstream parsing.

- Exact input file naming conventions for MS1/MS2 (with examples)
- The “RT reference” mechanism: what gets cached, when it is used, and safe ways to disable/rebuild it
- Output contract for the cohort tables (columns, blocks, and how to parse them reliably in R)

14. Retention-time (RT) reference system

EpiProfile-style layouts are RT-guided. For each peptide region, the code uses expected RTs to restrict the search window and to relocate peaks when signal is weak or crowded.

That mechanism is powerful, but it can also be a source of “silent failure” if you reuse old references across incompatible runs.

14.1 What the RT reference is

Conceptually, the RT reference is a small cache that stores “typical RT positions” for key peptides/features for a given LC method.

The workflow then uses those RTs to:

- define RT windows for XIC extraction,
- relocate peaks near expected RT,
- speed up processing and reduce false matching.

14.2 Where the RT reference is stored

In this repository build, RT reference artifacts are written under:

- `<raw_path>/histone_layouts/`

Common files you may encounter:

- `0_ref_info.mat` (global “reference info” for the folder)
- region-specific cached tables under per-run output folders (used by some layouts)

The important point is that the reference is “global by folder” (`raw_path`), not “global by project”.

14.3 When it is created and when it is reused

Typical behavior:

- If no reference exists, the first successful run may create one.
- If a reference exists, subsequent runs in the same `raw_path` may reuse it automatically.

This is convenient when you are processing many similar runs (same column, gradient, instrument, and general RT scale).

It is risky when you mix heterogeneous runs.

14.4 The main failure mode: cross-dataset contamination

If you process dataset A, then later reuse the same `raw_path` for dataset B (different LC method or instrument), the old RT reference can be applied to dataset B.

Symptoms:

- many peptides “missing” even though MS signal exists,
- RT windows fall outside the real chromatogram,
- relocation drives the search to the wrong place,
- in some cases: RT becomes 0 / empty windows / partial outputs.

14.5 Safe folder hygiene rules (recommended)

Use these rules unless you have a strong reason not to.

Rule 1 — one homogeneous batch per `raw_path`

Keep one `raw_path` per dataset batch that shares LC conditions.

Rule 2 — do not reuse `histone_layouts/` across incompatible runs

If you repurpose a folder, delete:

- `<raw_path>/histone_layouts/` entirely, or at minimum
- `<raw_path>/histone_layouts/0_ref_info.mat`

Rule 3 — avoid shared output folders between different run sets

Some layouts may write additional RT caches under per-run output folders. Reusing outputs can leak RT assumptions between runs.

14.6 Conservative mode (no RT reference)

If your goal is “no hidden assumptions”, run without reusing any cached RT reference.

Practical approach:

- use a fresh `raw_path`, or
- remove `histone_layouts/` before each run, or
- use the repository's "no reference / debug" mode if available in your build (some configs use an `ndebug` flag for this).

Document which approach you used in your run manifest.

15. Output contracts (what files mean)

This section describes the minimal set of outputs you should expect after a successful run, and how to treat them downstream.

15.1 Per-run folder structure

For each run, you should see:

- `<raw_path>/histone_layouts/01_<raw_name>/`
- `<raw_path>/histone_layouts/01_<raw_name>/detail/`

The `detail/` folder is where you will usually find:

- XIC plots per peptide region,
- identification tables / PSM summaries,
- intermediate exports used to build cohort-level matrices.

15.2 Cohort-level table: `histone_ratios.xls` (important)

Despite the extension, in this project `histone_ratios.xls` is often a TSV text file.

Treat it as text unless you have verified it is a real Excel binary file.

15.2.1 Typical structure (two header rows)

A common pattern is:

- Row 1: sample names (often repeated in blocks)
- Row 2: metric labels (e.g. `Ratio`, `Area`, `RT(min)`) plus a first-column label (often `Peptide` or `Feature`)
- Rows 3+: data, one row per feature (often peptideform / hPF)

Conceptual example (not literal):

(row2) Feature	SampleA Ratio	SampleA Area	SampleA RT(min)	SampleB Ratio	SampleB Area	SampleB RT(min)
H3_3_8 K4me1	...	...	...	...	...	...
H3_3_8 K4me2	...	...	...	...	...	...

Your downstream parser should not assume "one value per sample column". It must handle metric blocks.

15.3 Output contract: long-format table (recommended downstream)

For robust analysis, convert the cohort export to a long table with explicit keys.

Minimum recommended columns:

- **Dataset**
- **Sample**
- **Feature** (hPF identifier; exact string as in file)
- **Metric** (**Ratio**, **Area**, **RT(min)**)
- **Value**

Optional but strongly useful:

- **Peptide / Region** (e.g. **H3_3_8**)
- **Histone** (e.g. **H3**, **H4**, **H33**)
- **Modification** (parsed PTM string)
- **Sample_short / Condition / Group** (from your mapping file)
- **Ratio_area** (derived, if you keep both Ratio and Area)

16. Parsing **histone_ratios.xls** safely (R and Python)

This section gives a conservative parsing strategy that survives the “TSV disguised as XLS” pattern and the two-row header.

16.1 R (readr + tidyr) — conservative template

```
library(readr)
library(dplyr)
library(tidyr)
library(stringr)

path <- "histone_ratios.xls" # treat as TSV text

# Read raw as TSV without trusting header parsing
raw <- read_tsv(path, col_names = FALSE, show_col_types = FALSE)

# First two rows are headers
h1 <- raw[1, ] |> unlist(use.names = FALSE) |> as.character()
h2 <- raw[2, ] |> unlist(use.names = FALSE) |> as.character()

dat <- raw[-c(1,2), ]

# Name columns using a combined header: Sample + Metric
# Column 1 is the feature name
colnames(dat)[1] <- "Feature"

for (j in 2:ncol(dat)) {
  sample <- h1[j]
  metric <- h2[j]
  colnames(dat)[j] <- paste(sample, metric, sep = "__")
}

long <- dat |>
  pivot_longer(-Feature, names_to = "SampleMetric", values_to = "Value") |>
  separate(SampleMetric, into = c("Sample", "Metric"), sep = "__", extra = "merge") |>
```

```
mutate(Value = suppressWarnings(as.numeric(Value)))

# Now 'long' has Feature/Sample/Metric/Value
```

Notes:

- This assumes a 2-row header where row 1 has sample names and row 2 has metric labels.
- If your file uses a different pattern, adjust the header logic, not the downstream analysis.

16.2 Python (pandas) — conservative template

```
import pandas as pd

path = "histone_ratios.xls" # treat as TSV text

raw = pd.read_csv(path, sep="\t", header=None, dtype=str)

h1 = raw.iloc[0].tolist()
h2 = raw.iloc[1].tolist()

dat = raw.iloc[2:].copy()
dat.columns = ["Feature"] + [
    f"{h1[j]}_{h2[j]}" for j in range(1, raw.shape[1])
]

long = dat.melt(id_vars=["Feature"], var_name="SampleMetric", value_name="Value")
long[["Sample", "Metric"]] = long["SampleMetric"].str.split("_", n=1, expand=True)
long["Value"] = pd.to_numeric(long["Value"], errors="coerce")
long = long.drop(columns=["SampleMetric"])
```

17. Troubleshooting by symptom (quick)

17.1 “Everything is missing / outputs are mostly empty”

Most common causes:

- wrong `raw_names` (does not match file basenames)
- `ptol` too strict
- RT reference mismatch (old reference reused on incompatible runs)
- extraction failed (MS1/MS2 files incomplete or corrupted)

First actions:

- verify MS1/MS2 file pairs exist for each `raw_name`
- run 1 sample in isolation in a fresh `raw_path`
- remove `histone_layouts/` and rerun

17.2 “Some runs look fine, one run is almost blank”

Common causes:

- conversion/extraction failed for that run
- that run is a low-signal injection or wrong sample type
- file naming collision (two runs share a basename)

First actions:

- compare file sizes for MS1/MS2 across runs
- check XIC plots in [detail/](#) for a major peptide region
- confirm unique basenames

17.3 “RT looks wrong (shifted)”

Common causes:

- mixed LC methods in the same [raw_path](#)
- reused RT reference
- runs acquired on different columns/gradients

First actions:

- split runs into separate [raw_path](#) folders by LC method
- delete RT caches ([histone_layouts/](#)) and rerun

18. Reproducibility checklist (minimum)

For every dataset run, save:

- the exact commit hash (or release tag) of the repo
- the list/order of [raw_names](#)
- [raw_path](#) (and whether [histone_layouts/](#) was fresh)
- key parameters ([ptol](#), any debug/reference flags)
- the raw-to-mzML conversion method (tool + version)
- the MS1/MS2 extraction method (tool + version)
- a copy of the final cohort export(s) and QC summaries

If you cannot reconstruct these later, the run is not reproducible.

19. Next section to write

- The RT reference mechanism in more detail (what triggers creation/reuse per function, and the exact “safe disable” settings for this repository build).
- A formal “output schema” for the long table (required columns, allowed values, invariants, and validation checks).

20. RT reference mechanism (detailed)

This section documents the RT reference cache as it behaves in this repository build, and how to operate it safely.

20.1 Definitions

- **RT (retention time):** chromatographic time coordinate (often minutes) used to localize peptide elution.

- **Layout:** a curated set of expected peptides/peptideforms per histone region, with RT-aware extraction logic.
- **RT reference (cache):** a persisted set of RT anchors used to guide extraction and relocation across runs.

20.2 What “global by folder” means

In this build, RT reference artifacts are stored under:

- `<raw_path>/histone_layouts/`

The code checks this folder for a reference file (commonly `0_ref_info.mat`). If present, the reference can be applied to the run automatically.

Implication:

- If you reuse the same `raw_path` for a different dataset/instrument/LC method, you may accidentally reuse old RT anchors.

20.3 What gets cached

The exact content depends on the layout and function path, but the typical cached elements are:

- expected RTs (or RT windows) for key peptide regions,
- internal indices derived from RT windows (scan index ranges),
- sometimes additional per-region tables written during relocation.

Treat all cached RT artifacts as “stateful” and not portable between incompatible datasets.

20.4 When the cache helps (intended use)

RT caching is beneficial when you process:

- many runs with the same LC method,
- the same instrument family and comparable RT scale,
- stable chromatography (low drift).

In that context, the RT cache:

- reduces search space,
- improves robustness in crowded regions,
- speeds up quantification.

20.5 When the cache hurts (failure cases)

The cache can degrade results when:

- RT scale is different (short vs long gradients),
- column or LC method changed,
- instrument differs and chromatography shifts,
- you mix runs across acquisition days with large drift,
- you reuse an output folder across unrelated datasets.

Symptoms:

- systematic missingness,
- RT values are clipped or become 0,
- relocation points to wrong peaks,
- “empty window” behavior (no scans in the RT slice).

20.6 Operational modes (safe patterns)

Mode A — Stable batch mode (reuse allowed)

Use this when:

- all runs are homogeneous,
- you want maximum throughput.

Rules:

- keep a single `raw_path` for the batch,
- do not move unrelated runs into the folder later,
- archive the full folder after processing (including `histone_layouts/`).

Mode B — Conservative mode (no reuse)

Use this when:

- you are validating a new dataset,
- you are cross-instrument,
- you are debugging missingness.

Rules:

- use a fresh `raw_path`, or
- delete `histone_layouts/` before each run set, at least:
 - `<raw_path>/histone_layouts/0_ref_info.mat`

This prevents “silent carryover”.

Mode C — Split-by-method mode (recommended default)

Use this when you have multiple LC methods or instruments.

Rules:

- one `raw_path` per homogeneous group, e.g.
 - `.../MS1_MS2/PXD046034_ZenoTOF/`
 - `.../MS1_MS2/PXD046034_Orbitrap/`
- never share `histone_layouts/` between those groups.

20.7 Minimal documentation to record

In your run manifest, record:

- whether `histone_layouts/` was created fresh or reused,
- whether `0_ref_info.mat` existed at start,

- the grouping rule used (by PXD, by instrument, by LC method).

If you cannot state these later, RT behavior is not auditable.

21. Output schema (long-format contract)

Downstream analysis should operate on a long-format table with explicit keys. This makes parsing robust across:

- two-row headers,
- metric blocks (Ratio/Area/RT),
- varying column orders,
- new metrics added later.

21.1 Required columns

A minimal long table MUST contain:

- **Dataset**
Identifier of the dataset run context (e.g. `PXD046034`, `ONTOGENY_2025`).
- **Sample**
The sample/run name as exported (matches header sample name in `histone_ratios`).
- **Feature**
The feature identifier as exported by MATLAB (often a peptideform / hPF string).
- **Metric**
One of: `Ratio`, `Area`, `RT(min)` (or an allowed extension; see below).
- **Value**
Numeric value (float). Missing values must be `NA/NaN`, not empty strings.

21.2 Recommended derived columns (strongly suggested)

These are not mandatory for a parser, but make analysis reproducible:

- **Peptide or Region**
Parsed histone region identifier (e.g. `H3_3_8`, `H4_4_17`).
- **Histone**
Parsed histone class (e.g. `H3`, `H33`, `H4`).
- **Modification**
Parsed PTM string (free text, but stable conventions recommended).
- **Sample_short**
A short sample ID (often numeric prefix stripped).
- **Condition / Group**
Biological grouping used for contrasts.
- **Contrast**
Optional explicit contrast label (e.g. `SEN_vs_YNG`).

21.3 Allowed values and invariants

Metric invariants

- **Ratio** must be in $[0, 1]$ for compositional ratios, unless your build defines a different scale.
- **Area** must be ≥ 0 .
- **RT(min)** must be ≥ 0 and usually within the LC gradient duration.

If these invariants are violated systematically, treat it as a pipeline issue, not a biology issue.

Feature uniqueness

In long form, a single cell is uniquely defined by:

- (**Dataset**, **Sample**, **Feature**, **Metric**)

This key must be unique. Duplicates indicate:

- repeated columns in export,
- name collisions,
- parsing errors.

21.4 Validation checks (minimum)

Before downstream statistics, run these checks:

- **Key uniqueness:** no duplicates on (**Dataset**, **Sample**, **Feature**, **Metric**)
- **Metric coverage:** each (**Dataset**, **Sample**, **Feature**) has expected metrics present
- **Range checks:** apply invariants above
- **Missingness summary:** per-sample fraction of missing **Ratio** and **Area**
- **Sanity anchors:** pick 2–3 abundant peptide regions and confirm non-zero areas across most samples

If any check fails, stop and fix parsing or upstream issues.

22. Minimal QC outputs (what to keep)

For each dataset, keep a small QC packet:

- total number of detected features per sample (count of non-missing **Area**)
- distribution of $\log_{10}(\text{Area})$ per sample
- PCA on **Ratio** (after basic filtering)
- heatmap of selected single-PTM summaries (optional but useful)
- a short list of “anchor peptides” with their RT across samples

Do not skip QC even if the run “finished”.

23. Run manifest template (copy/paste)

Create a plain text file per dataset run, for example:

- `runs/PXD046034_run_manifest.txt`

Template:

- Dataset:
- Date:
- Repo commit:
- raw_path:
- raw_names (ordered list):
- ptol:
- Conversion tool + version:
- Extraction tool + version:
- RT reference:
 - histone_layouts existed before run? (yes/no)
 - 0_ref_info.mat existed before run? (yes/no)
 - reused or fresh? (reused/fresh)
- Notes / anomalies:

This file is often the difference between “reproducible” and “lost”.

24. Next section to write

- Formal mapping rules: parsing **Feature** → (**Histone**, **Region**, **PTM**) with include/exclude patterns and a validator.
- A reference set of “anchor peptides” per species/build to standardize QC across datasets.
- A minimal example dataset walkthrough (one run) with expected folder tree and expected key outputs.

25. Feature parsing and mapping rules

This section defines a conservative way to turn the exported **Feature** strings into structured fields. The goal is not “perfect semantics”, but “stable parsing” that supports reproducible summaries.

25.1 Why you need parsing

The MATLAB exports typically encode, in one string:

- histone region (e.g. **H3_27_40**)
- peptide sequence (sometimes)
- PTM annotations (often as tokens like **K27me2**, **K14ac**, etc.)
- sometimes variant labels (e.g. **H33** vs **H3**)
- sometimes extra text (layout-specific)

Downstream, you will want to:

- aggregate peptideforms to peptide-level ratios (hPF → hDP),
- aggregate peptideforms to mark-level summaries (e.g. “all forms containing K27me3”),
- validate that ambiguous forms are not misassigned into the wrong mark bucket.

25.2 Minimal parsing outputs

From each **Feature** string, try to extract:

Required (minimum useful set):

- **Region** (e.g. H3_27_40, H4_4_17)
- **Histone** (e.g. H3, H33, H4)
- **PTM_tokens** (list of PTM tokens found in the string)

Optional (if available and consistent):

- **PeptideSequence**
- **IsVariant** / **VariantLabel**

25.3 Conservative region extraction

In most EpiProfile-style layouts, region tokens look like:

- H3_3_8
- H3_27_40
- H33_27_40
- H4_4_17

Conservative regex (language-agnostic):

- `H(3|33|4)_{d+}_{d+}`

Rule:

- take the first match as **Region**
- derive **Histone** from the prefix (H3, H33, H4)

If no region is detected, do not “guess”. Mark as **Region** = **NA** and handle explicitly.

25.4 PTM token extraction (basic)

A practical PTM token pattern for histone marks is:

- residue letter + position + modification label

Examples:

- K4me1, K4me2, K4me3
- K14ac
- S10ph
- K27me3
- K36me2

Conservative regex:

- `[KRSY]\d+(me[123]|ac|ph)`

Rule:

- extract all matches as **PTM_tokens**
- keep them as strings, do not interpret beyond tokenization at this step

25.5 Mark aggregation using include/exclude patterns

When you build mark-level summaries (“all peptideforms carrying X”), use explicit include/exclude rules. This avoids common errors like accidentally counting **K27me3** into **K27me2**.

Example: define a mark as a tuple:

- **include**: pattern(s) that must be present
- **exclude**: pattern(s) that must not be present

Example rules:

- Mark: **H3K27me2**
 - include: **K27me2**
 - exclude: **K27me3**, **K27me1**, **K27ac**
- Mark: **H3K27ac**
 - include: **K27ac**
 - exclude: **K27me1**, **K27me2**, **K27me3**
- Mark: **H3S10ph**
 - include: **S10ph**
 - exclude: (usually none, unless your feature strings include ambiguous labels)

Do not rely on substring logic like “contains K27”. Always include the full token.

25.6 Validator: catch inconsistent assignments

Before you accept a mark-level summary, validate the membership set.

For each mark rule:

1. list all features included by the rule
2. scan each included feature for forbidden tokens
3. report violations as an error (not a warning)

Minimum checks:

- **K27me2** summary must contain zero features with **K27me3**
- **K4me3** summary must contain zero features with **K4me2** or **K4me1**
- acetylation summaries must not contain methylation tokens at the same residue (unless your biology definition explicitly allows “any K27 state”, which should be a different summary name)

If the validator fails, fix the rule set, not the data.

26. Anchor peptides for QC

Anchors are peptide regions you expect to see in most runs if the pipeline is healthy. They are used for quick sanity checks and RT drift inspection.

26.1 What makes a good anchor

- strong signal in typical histone prep

- low ambiguity
- present across many samples
- stable elution behavior

26.2 How to use anchors

For each dataset:

- pick 3–5 anchor regions
- for each sample, record:
 - `Area` (or `log10 Area`)
 - `RT(min)`
- plot or summarize:
 - missingness per anchor
 - RT median and IQR per anchor
 - samples with outlier RT drift or near-zero areas

A dataset can “finish” but still fail anchor QC.

27. Minimal end-to-end walkthrough (one run)

This is the smallest useful run you can do to validate an installation and folder hygiene.

27.1 Prepare one run

- Choose one LC–MS/MS run with good signal.
- Ensure you have MS1/MS2 extracted files in:
 - `<raw_path>/`
- Ensure the basename is stable and clean:
 - `raw_name = "Run_001"` (example)

27.2 Run MATLAB

- Start with a fresh output state:
 - delete `<raw_path>/histone_layouts/` if it exists
- Run:
 - `DrawISOProfile1(raw_path, {raw_name}, ptol, special)`

27.3 Verify outputs

You should see:

- `<raw_path>/histone_layouts/01_Run_001/`
- `<raw_path>/histone_layouts/01_Run_001/detail/`

Then confirm:

- at least some XIC plots exist in `detail/`
- at least one cohort export exists (even if only one sample)

27.4 Parse the cohort export (text-first)

If you get `histone_ratios.xls`:

- open it as text (TSV)
- confirm:
 - two header rows
 - metric blocks (Ratio/Area/RT)
 - feature rows are non-empty

Then parse into long form (R or Python templates).

If you cannot parse it deterministically, stop and fix parsing before scaling to many runs.

28. File mapping (samples → conditions)

Downstream analysis needs a mapping file. Keep it explicit and versioned.

Minimum columns:

- `Sample` (exactly matches export header)
- `Condition` (e.g. `YNG`, `BOT`, `FLOR`, `SEN`)
- `Group` (optional, e.g. `control`, `treated`)
- `Batch` (optional, if you use ComBat or similar)
- `Order` (optional, injection order)

Rules:

- `Sample` must be unique
 - missing mappings are errors, not warnings
 - never infer condition labels from sample names unless the mapping file says so
-

29. Next section to write

- A strict definition of hDP, hPF, and hPTM in this repository context.
- A canonical list of marks to summarize (with include/exclude rules) for the default Arabidopsis layouts.
- A small “QC decision tree”: what to do when missingness, RT drift, or intensity collapse is observed.

30. Core data model (hDP, hPF, hPTM)

This section defines the three levels that EpiProfile_PLANTS uses implicitly in exports and downstream analysis. The definitions here are operational: they describe how you should treat rows and aggregates in a reproducible pipeline.

30.1 hDP — histone-derived peptide (peptide level)

An **hDP** is a peptide region defined by sequence and boundaries on the histone (e.g. `H3_27_40`, `H4_4_17`). It is the “parent” entity.

Properties:

- one hDP can generate many modified forms (hPFs)
- hDP-level summaries are usually computed by aggregating over its child hPFs

In practice:

- an hDP corresponds to a region token like `H3_27_40` plus (optionally) a specific sequence variant label.

30.2 hPF — histone peptideform (modified form level)

An **hPF** is one specific modified state of an hDP. It is typically what appears as a **row** in `histone_ratios` exports.

Examples (conceptual):

- `H3_27_40 K27me2`
- `H3_27_40 K27me3`
- `H4_4_17 K5acK8acK12ac`

Properties:

- hPFs are compositional within an hDP (many builds enforce that the ratios across hPFs within the same hDP sum to ~1 per sample)
- hPFs carry the PTM tokens used for mark-level aggregation

In practice:

- treat each exported row as an hPF unless you have explicit evidence it is already an hDP.

30.3 hPTM — mark summary (marginal PTM level)

An **hPTM** is a derived metric: the marginal abundance of a single mark (or mark family) aggregated across all hPFs that contain the mark.

Example:

- `H3K27me3` summary = sum of ratios of all hPFs that contain token `K27me3` (within the relevant hDP or across multiple hDPs, depending on definition)

Important:

- hPTMs require explicit include/exclude rules (Section 25.5–25.6)
- a mark summary is only meaningful if the rule set is stable and validated

30.4 Where ratios live (composition semantics)

Typical semantics in this project:

- **Area** is an absolute-ish signal proxy per hPF (within-run intensity scale).
- **Ratio** is a normalized composition within a peptide region:
 - $\text{Ratio}(\text{hPF}) = \text{Area}(\text{hPF}) / \text{sum Area}(\text{all hPFs of that hDP})$ for that sample.

This implies:

- ratios are comparable across samples for the same hDP,
- areas are useful for QC and filtering, but not directly comparable across runs without normalization.

If your export is not strictly compositional, document the difference explicitly in the run manifest and downstream scripts.

31. Default mark list (Arabidopsis-oriented)

This section proposes a practical default list of marks to summarize for Arabidopsis H3/H4 layouts. Treat it as a starting point; your layout may not cover all items.

31.1 H3 / H3.3 marks (typical)

- H3K4me1, H3K4me2, H3K4me3, H3K4ac
- H3K9me1, H3K9me2, H3K9me3, H3K9ac
- H3S10ph, H3S10ac (if present)
- H3K14ac
- H3K18me1, H3K18ac
- H3K23me1, H3K23ac
- H3K27me1, H3K27me2, H3K27me3, H3K27ac
- H3S28ph, H3S28ac (if present)
- H3K36me1, H3K36me2, H3K36me3
- H3Y41ph (if present)
- H3K56me1, H3K56me2, H3K56me3, H3K56ac
- H3K79me1, H3K79me2, H3K79me3, H3K79ac
- H3K122ac (if present)

Notes:

- Keep H3.1 vs H3.3 separate if your region tokens distinguish them (H3_... vs H33_...).
- Do not merge K27me3 with K27me2 “because both are K27 methylation”. If you need that, define an explicit combined metric (e.g. H3K27me2_3).

31.2 H4 marks (typical)

- H4K5ac, H4K8ac, H4K12ac, H4K16ac (and multi-acetyl combinations via marginal sums)
- H4K20me1, H4K20me2, H4K20me3, H4K20ac (if present)
- H4K44ac (if present)

31.3 Example include/exclude rules (pattern-based)

Examples (conceptual; implement in code as regex rules):

- H3K27me2:
 - include: K27me2
 - exclude: K27me1, K27me3, K27ac
- H4K16ac:

- include: **K16ac**
- exclude: none (unless your strings include ambiguous artifacts)
- **H3K36me3**:
 - include: **K36me3**
 - exclude: **K36me1**, **K36me2**

Always validate the membership sets.

32. Filtering strategy (minimum recommended)

You need filtering before statistics to avoid “phantom biology” driven by missingness and low-intensity artifacts.

32.1 Feature-level filters (hPF level)

Apply at least:

- remove hPFs with **Area** missing in too many samples (choose a threshold; common: keep features present in ≥ 70 –80% of samples)
- remove hPFs with near-zero **Area** across most samples (if your export uses zeros)

Keep the filtering rule written down.

32.2 Peptide-level filters (hDP level)

For hDP-level summaries, also track:

- fraction of samples where the **total peptide area** is non-missing
- peptides with unstable detection should be excluded from mark summaries that depend on them

32.3 Mark-level filters (hPTM level)

For each mark summary:

- report how many hPFs contribute
 - report how many samples have valid values after filtering
 - never interpret a mark that is supported by a tiny, inconsistent contributor set
-

33. QC decision tree (minimal)

Use this to decide what to fix first when something looks wrong.

33.1 If missingness is high in many samples

Likely upstream:

- conversion/extraction problem
- wrong naming / wrong pairing
- RT cache contamination

Actions:

- isolate 1–2 runs and rerun in a fresh `raw_path`
- inspect anchor peptide XICs
- delete `histone_layouts/` and rerun

33.2 If one sample is an outlier in intensity

Likely:

- injection issue
- wrong file (blank or different sample)
- partial extraction

Actions:

- compare MS1/MS2 file sizes
- inspect anchor peptide areas
- consider excluding the sample if technically failed

33.3 If RT drift is systematic

Likely:

- mixed LC methods
- wrong reference reuse
- gradient mismatch

Actions:

- split by method/instrument
- fresh RT cache per group

33.4 If ratios look “flat” (no dynamics anywhere)

Likely:

- ratio block mis-parsed
- columns misaligned due to header handling
- you are using `Area` when you think you are using `Ratio` (or vice versa)

Actions:

- validate parsing with a small subset
- confirm Metric labels (`Ratio` vs `Area`) are correct
- run key uniqueness checks and range checks

34. Next section to write

- A complete worked example:
 - one `histone_ratios` file,
 - parsing to long format,
 - computing hDP-normalized ratios,
 - computing hPTM marginal sums with validation,
 - generating a minimal QC report (counts, intensity boxplot, PCA, heatmap).

35. Worked example: from `histone_ratios` to hDP/hPTM + minimal QC

This section shows a complete, conservative workflow using a single cohort export (often named `histone_ratios.xls`, but treated as TSV text). The goal is reproducibility, not cleverness.

You will:

1. parse the export into long format,
2. extract `Region/Histone/PTM_tokens`,
3. compute hDP-normalized ratios (if needed),
4. compute hPTM marginal sums with include/exclude rules + validation,
5. generate minimal QC summaries.

35.1 Inputs

Required:

- `histone_ratios.xls` (TSV-like text with 2 header rows)
- optional `sample_map.tsv` with columns like `Sample`, `Condition`, `Batch`, etc.

Assumptions in this worked example:

- Row 1 of the file contains sample names (repeated across metric blocks).
- Row 2 contains metric names (e.g. `Ratio`, `Area`, `RT(min)`).
- Column 1 contains `Feature` identifiers (hPF rows).

If your export differs, adjust header parsing only. Do not change downstream analysis logic.

35.2 R worked example (readr/tidyr)

35.2.1 Parse TSV-disguised export to long format

```
library(readr)
library(dplyr)
library(tidyr)
library(stringr)

path <- "histone_ratios.xls" # treat as TSV text

raw <- read_tsv(path, col_names = FALSE, show_col_types = FALSE)

h1 <- raw[1, ] |> unlist(use.names = FALSE) |> as.character()
h2 <- raw[2, ] |> unlist(use.names = FALSE) |> as.character()
```

```

dat <- raw[-c(1,2), ] |> as.data.frame(stringsAsFactors = FALSE)
colnames(dat)[1] <- "Feature"

for (j in 2:ncol(dat)) {
  sample <- h1[j]
  metric <- h2[j]
  colnames(dat)[j] <- paste(sample, metric, sep = "__")
}

long <- dat |>
  pivot_longer(-Feature, names_to = "SampleMetric", values_to = "Value") |>
  separate(SampleMetric, into = c("Sample", "Metric"), sep = "__", extra = "merge") |>
  mutate(Value = suppressWarnings(as.numeric(Value)))

# Keep only the metrics you will use
long <- long |> filter(Metric %in% c("Ratio", "Area", "RT(min)"))

```

35.2.2 Add structured parsing: Region, Histone, PTM tokens

```

parse_region <- function(feature) {
  m <- str_match(feature, "(H(3|33|4)_\\d+_\\d+)")
  m[,2]
}

parse_histone <- function(region) {
  # region like "H33_27_40" -> "H33"
  ifelse(is.na(region), NA_character_, str_extract(region, "^H(3|33|4)"))
}

parse_ptm_tokens <- function(feature) {
  # returns a single string with tokens separated by ";"
  toks <- str_extract_all(feature, "([KRSY]\\d+(me[123]|ac|ph))")[[1]]
  if (length(toks) == 0) NA_character_ else paste(unique(toks), collapse = ";")
}

long2 <- long |>
  mutate(
    Region = parse_region(Feature),
    Histone = parse_histone(Region),
    PTM_tokens = vapply(Feature, parse_ptm_tokens, character(1))
  )

```

35.2.3 Compute hDP-level totals and (re)derive compositional ratios if needed

This is conservative because exports may include both **Area** and **Ratio**. You can compute a fresh ratio from **Area** and compare.

```

wide <- long2 |>
  pivot_wider(names_from = Metric, values_from = Value)

# Peptide total area per sample and region (hDP proxy)
pep_tot <- wide |>
  filter(!is.na(Region)) |>
  group_by(Sample, Region) |>
  summarise(PeptideTotalArea = sum(Area, na.rm = TRUE), .groups = "drop") |>
  mutate(PeptideTotalArea = ifelse(PeptideTotalArea == 0, NA_real_, PeptideTotalArea))

wide2 <- wide |>
  left_join(pep_tot, by = c("Sample", "Region")) |>
  mutate(
    Ratio_from_Area = Area / PeptideTotalArea
  )

```

```
# Optional check: compare provided Ratio vs recomputed Ratio_from_Area
ratio_check <- wide2 |>
  filter(!is.na(Ratio), !is.na(Ratio_from_Area)) |>
  summarise(
    corr = cor(Ratio, Ratio_from_Area),
    mean_abs_diff = mean(abs(Ratio - Ratio_from_Area))
  )
print(ratio_check)
```

Interpretation:

- High correlation and low absolute difference suggests your parsing and semantics are aligned.
- If correlation is low, suspect header misalignment (wrong Metric labels) or different ratio definition.

35.2.4 Define mark rules and compute hPTM marginal sums

Define rules as a small table. Include/exclude patterns must be explicit.

```
mark_rules <- tibble::tribble(
  ~Mark,      ~Include,  ~Exclude,
  "H3K27me1", "K27me1",  "K27me2;K27me3;K27ac",
  "H3K27me2", "K27me2",  "K27me1;K27me3;K27ac",
  "H3K27me3", "K27me3",  "K27me1;K27me2;K27ac",
  "H3K27ac",  "K27ac",   "K27me1;K27me2;K27me3",
  "H4K16ac",  "K16ac",   NA_character_
)

has_token <- function(ptm_tokens, token) {
  ifelse(is.na(ptm_tokens), FALSE, str_detect(ptm_tokens, paste0("(^|;)", token, "($|;)")))
}

violates_exclude <- function(ptm_tokens, excl_string) {
  if (is.na(excl_string) || is.na(ptm_tokens)) return(FALSE)
  excl <- str_split(excl_string, ";")[[1]]
  any(vapply(excl, function(t) has_token(ptm_tokens, t), logical(1)))
}

# Build membership and validate
membership <- wide2 |>
  filter(!is.na(Region)) |>
  select(Sample, Feature, Region, Histone, PTM_tokens, Ratio_from_Area)

compute_mark <- function(df, mark, include, exclude) {
  df_in <- df |>
    mutate(included = has_token(PTM_tokens, include),
           excluded = vapply(PTM_tokens, violates_exclude, logical(1), excl_string = exclude)) |>
    filter(included)

  # Validator: no excluded tokens allowed in the included set
  bad <- df_in |> filter(excluded)
  if (nrow(bad) > 0) {
    stop(paste0("Validator failed for ", mark, ": included features contain excluded tokens."))
  }

  df_in |>
    group_by(Sample) |>
    summarise(Value = sum(Ratio_from_Area, na.rm = TRUE), .groups = "drop") |>
    mutate(Mark = mark)
}

hptm <- purrr::pmap_dfr(
  list(mark_rules$Mark, mark_rules$Include, mark_rules$Exclude),
  function(Mark, Include, Exclude) compute_mark(membership, Mark, Include, Exclude)
)

# hPTM is a long table: Sample / Mark / Value
```

35.2.5 Minimal QC summaries (counts, intensity scale, missingness)

```
qc_counts <- wide2 |>
  filter(!is.na(Area)) |>
  group_by(Sample) |>
  summarise(
    n_features_area = sum(!is.na(Area)),
    n_features_ratio = sum(!is.na(Ratio_from_Area)),
    .groups = "drop"
  )

qc_missing <- wide2 |>
  filter(!is.na(Region)) |>
  group_by(Sample) |>
  summarise(
    frac_missing_area = mean(is.na(Area)),
    frac_missing_ratio = mean(is.na(Ratio_from_Area)),
    .groups = "drop"
  )

print(qc_counts)
print(qc_missing)
```

Optional plots (keep simple, no styling assumptions):

- boxplot of `log10(Area)` by sample
- RT distribution of 2–3 anchor regions

35.3 Python worked example (pandas)

35.3.1 Parse TSV-disguised export to long format

```
import pandas as pd
import re

path = "histone_ratios.xls" # treat as TSV text
raw = pd.read_csv(path, sep="\t", header=None, dtype=str)

h1 = raw.iloc[0].tolist()
h2 = raw.iloc[1].tolist()

dat = raw.iloc[2:].copy()
dat.columns = ["Feature"] + [f"{h1[j]}__{h2[j]}" for j in range(1, raw.shape[1])]

long = dat.melt(id_vars=["Feature"], var_name="SampleMetric", value_name="Value")
long[["Sample", "Metric"]] = long["SampleMetric"].str.split("__", n=1, expand=True)
long["Value"] = pd.to_numeric(long["Value"], errors="coerce")
long = long.drop(columns=["SampleMetric"])

long = long[long["Metric"].isin(["Ratio", "Area", "RT(min)])].copy()
```

35.3.2 Parse Region/Histone/PTM tokens

```
region_re = re.compile(r"(H(3|33|4)_\d+_\d+)")
ptm_re = re.compile(r"([KRSY]\d+(?:me[123]|ac|ph))")

def parse_region(feature: str):
```

```

if feature is None:
    return None
m = region_re.search(feature)
return m.group(1) if m else None

def parse_histone(region: str):
    if region is None:
        return None
    return region.split("_")[0] # H3, H33, H4

def parse_ptm_tokens(feature: str):
    if feature is None:
        return None
    toks = ptm_re.findall(feature)
    if not toks:
        return None
    toks = sorted(set(toks))
    return ";".join(toks)

long["Region"] = long["Feature"].map(parse_region)
long["Histone"] = long["Region"].map(parse_histone)
long["PTM_tokens"] = long["Feature"].map(parse_ptm_tokens)

```

35.3.3 Pivot to wide and compute Ratio from Area

```

wide = long.pivot_table(
    index=["Sample", "Feature", "Region", "Histone", "PTM_tokens"],
    columns="Metric",
    values="Value",
    aggfunc="first"
).reset_index()

# Peptide total area per sample+region
pep_tot = (
    wide.dropna(subset=["Region"])
    .groupby(["Sample", "Region"], as_index=False)["Area"]
    .sum()
    .rename(columns={"Area": "PeptideTotalArea"})
)

pep_tot.loc[pep_tot["PeptideTotalArea"] == 0, "PeptideTotalArea"] = pd.NA

wide2 = wide.merge(pep_tot, on=["Sample", "Region"], how="left")
wide2["Ratio_from_Area"] = wide2["Area"] / wide2["PeptideTotalArea"]

# Optional: compare to provided Ratio
ratio_check = wide2.dropna(subset=["Ratio", "Ratio_from_Area"])
if len(ratio_check) > 2:
    corr = ratio_check["Ratio"].corr(ratio_check["Ratio_from_Area"])
    mean_abs_diff = (ratio_check["Ratio"] - ratio_check["Ratio_from_Area"]).abs().mean()
    print({"corr": corr, "mean_abs_diff": mean_abs_diff})

```

35.3.4 Mark rules + validator + marginal sums

```

def has_token(token_string, token):
    if token_string is None or pd.isna(token_string):
        return False
    parts = token_string.split(";")
    return token in parts

def violates_exclude(token_string, exclude_str):
    if exclude_str is None or pd.isna(exclude_str) or token_string is None or pd.isna(token_string):
        return False
    excl = exclude_str.split(";")
    return any(has_token(token_string, t) for t in excl)

mark_rules = [

```

```

{"Mark": "H3K27me2", "Include": "K27me2", "Exclude": "K27me1;K27me3;K27ac"},
{"Mark": "H3K27me3", "Include": "K27me3", "Exclude": "K27me1;K27me2;K27ac"},
{"Mark": "H3K27ac", "Include": "K27ac", "Exclude": "K27me1;K27me2;K27me3"},
]

rows = []
for rule in mark_rules:
    df = wide2.dropna(subset=["Region"]).copy()
    df["included"] = df["PTM_tokens"].map(lambda s: has_token(s, rule["Include"]))
    df["excluded"] = df["PTM_tokens"].map(lambda s: violates_exclude(s, rule["Exclude"]))

    df_in = df[df["included"].copy()]

    # Validator: no excluded tokens in included set
    bad = df_in[df_in["excluded"]]
    if len(bad) > 0:
        raise RuntimeError(f"Validator failed for {rule['Mark']}: excluded tokens present in included set.")

    out = (
        df_in.groupby("Sample", as_index=False)["Ratio_from_Area"]
        .sum()
        .rename(columns={"Ratio_from_Area": "Value"})
    )
    out["Mark"] = rule["Mark"]
    rows.append(out)

hptm = pd.concat(rows, ignore_index=True)

```

35.3.5 Minimal QC summaries

```

qc_counts = (
    wide2.groupby("Sample", as_index=False)
    .agg(
        n_features_area=("Area", lambda x: x.notna().sum()),
        n_features_ratio=("Ratio_from_Area", lambda x: x.notna().sum()),
    )
)

qc_missing = (
    wide2.dropna(subset=["Region"])
    .groupby("Sample", as_index=False)
    .agg(
        frac_missing_area=("Area", lambda x: x.isna().mean()),
        frac_missing_ratio=("Ratio_from_Area", lambda x: x.isna().mean()),
    )
)

print(qc_counts)
print(qc_missing)

```

If QC suggests a technical failure, stop and fix upstream steps (conversion/extraction/naming/RT reference) before any statistics.

36. Packaging recommendations (keep the workflow auditable)

This repository is easiest to maintain if you keep analysis as small, named scripts with explicit inputs/outputs.

Suggested structure:

- `analysis/`

- `01_parse_histone_ratios.R` or `.py`
- `02_build_long_table.R` or `.py`
- `03_compute_hdp_hptm.R` or `.py`
- `04_qc_report.R` or `.py`
- `configs/`
 - `mark_rules.tsv` (Mark / Include / Exclude)
 - `sample_map.tsv`
- `runs/`
 - `PXDxxxxxx_run_manifest.txt`

Rules:

- never hardcode paths inside functions; pass them as variables or CLI args
 - version `mark_rules.tsv` and `sample_map.tsv` with the dataset
 - save intermediate long tables (`.parquet` or `.tsv`) so you can re-run later without re-parsing headers
-

37. Next section to write

- A strict, repository-specific definition of the canonical feature naming scheme (what a “Feature” string must contain).
- A small set of invariant tests (unit-test style) that fail fast if:
 - header parsing is wrong,
 - Metric labels are misread,
 - key uniqueness is violated,
 - ratio semantics drift across versions.

38. Canonical **Feature** string specification (practical contract)

This repository treats the exported **Feature** field as the stable identifier for a peptideform (hPF). Because layouts can vary, the goal here is a *minimal contract* that downstream parsing can rely on.

38.1 Minimal required elements

A valid **Feature** string SHOULD contain:

1. A region token (required for structured analysis)

Format:

- `H3_<start>_<end>`
- `H33_<start>_<end>`
- `H4_<start>_<end>`

Examples:

- `H3_27_40`
- `H33_27_40`
- `H4_4_17`

2. Zero or more PTM tokens (optional, but usually present)

Format:

- residue + position + label
- `K27me3`, `K14ac`, `S10ph`, `Y41ph`, etc.

3. A stable text remainder (optional)

This can include peptide sequence, charge labels, or layout-specific descriptors. Downstream analysis should not depend on the exact content of this remainder.

If no region token exists, the row is still allowed to exist, but it must be handled explicitly as `Region = NA`.

38.2 Allowed PTM token vocabulary (recommended)

To keep aggregation stable, prefer these token patterns:

- methylation: `me1`, `me2`, `me3`
- acetylation: `ac`
- phosphorylation: `ph`

Examples:

- `K4me1`, `K4me2`, `K4me3`
- `K27ac`
- `S10ph`

If your build uses additional labels (e.g. `pr` for propionyl), treat them as “extra tokens” and do not mix them into mark summaries unless you define rules.

38.3 Separator conventions (do not rely on them)

Feature strings may use:

- spaces,
- commas,
- semicolons,
- parentheses.

Downstream parsing MUST use regex token extraction, not strict splitting on separators.

38.4 Recommended policy for new layouts

If you create or modify layouts, keep these invariants:

- region token present and consistent
- PTM token format consistent
- do not embed sample-specific information inside `Feature`

39. Invariant checks (fail fast)

You want errors early, not subtle problems later. These checks should run after parsing and before any biological interpretation.

39.1 Parsing invariants (table-level)

Given a long table with columns:

- `Dataset`, `Sample`, `Feature`, `Metric`, `Value`

Check:

1. Key uniqueness

- no duplicates on (`Dataset`, `Sample`, `Feature`, `Metric`)

2. Allowed metrics

- `Metric` in `{Ratio, Area, RT(min)}` (or your declared extension set)

3. Numeric coercion

- `Value` is numeric for all metrics (missing values allowed)

4. Non-negativity

- `Area` ≥ 0
- `RT(min)` ≥ 0

39.2 Composition invariants (hDP-level)

After you compute `Region` and a ratio (either provided or `Ratio_from_Area`):

For each (`Sample`, `Region`):

- sum of ratios across hPFs is approximately 1.0 (allow tolerance)

Suggested tolerance (practical):

- flag if sum < 0.95 or sum > 1.05

If this fails broadly:

- you may have parsing misalignment, or
- the export is not compositional (must be documented).

39.3 Mark-rule invariants (hPTM validator)

For each mark rule:

- included features must not contain excluded tokens (hard error)

This is not optional. If the validator fails, mark summaries are not trustworthy.

40. Minimal tests (R and Python)

These are “small scripts that fail with a clear message”. You can later wrap them in test frameworks.

40.1 R (base + stopifnot) — minimal checks

```
# inputs:
# long_df: Dataset, Sample, Feature, Metric, Value
# parsed_df: adds Region, Ratio_from_Area, Area, etc.

library(dplyr)

assert_unique_key <- function(df) {
  dup <- df |>
    count(Dataset, Sample, Feature, Metric) |>
    filter(n > 1)
  if (nrow(dup) > 0) stop("Key uniqueness failed: duplicated (Dataset,Sample,Feature,Metric).")
}

assert_allowed_metrics <- function(df, allowed = c("Ratio","Area","RT(min)")) {
  bad <- setdiff(unique(df$Metric), allowed)
  if (length(bad) > 0) stop(paste("Unexpected Metric values:", paste(bad, collapse=" ")))
}

assert_nonneg <- function(df) {
  if (any(df$Metric == "Area" & df$Value < 0, na.rm = TRUE)) stop("Negative Area detected.")
  if (any(df$Metric == "RT(min)" & df$Value < 0, na.rm = TRUE)) stop("Negative RT(min) detected.")
}

assert_ratio_sums <- function(wide_df, tol_low = 0.95, tol_high = 1.05) {
  # wide_df must have: Sample, Region, Ratio_from_Area
  sums <- wide_df |>
    filter(!is.na(Region)) |>
    group_by(Sample, Region) |>
    summarise(sum_ratio = sum(Ratio_from_Area, na.rm = TRUE), .groups = "drop")

  bad <- sums |> filter(sum_ratio < tol_low | sum_ratio > tol_high)
  if (nrow(bad) > 0) {
    stop("Ratio sum invariant failed for at least one (Sample,Region).")
  }
}

# Usage:
# assert_unique_key(long_df)
# assert_allowed_metrics(long_df)
# assert_nonneg(long_df)
# assert_ratio_sums(wide2)
```

45. Expected folder tree (reference example)

This section shows what a “healthy” dataset looks like on disk. The exact filenames vary, but the structure should be recognizable.

45.1 Dataset root (public PXD example)

- PX_DATA/
 - PXDxxxxxx/
 - raw/
 - raw_thermo/ or wiff/
 - Run_001.RAW or Run_001.wiff + Run_001.wiff.scan
 - mzML/
 - Run_001.mzML
 - MS1_MS2/

- Run_001_MS1.txt
- Run_001_MS2.txt
- epiprofile_output/ (optional, if you choose to stage outputs here)
- metadata/
 - sample_map.tsv
 - run_manifest.txt
 - mark_rules.tsv
- logs/
 - conversion logs
 - extraction logs

45.2 MATLAB output folder (created under **raw_path**)

If `raw_path = .../PX_DATA/PXDxxxxxx/MS1_MS2`, then MATLAB creates:

- .../PX_DATA/PXDxxxxxx/MS1_MS2/histone_layouts/
 - 0_ref_info.mat (may exist depending on mode)
 - 01_Run_001/
 - detail/
 - XIC plots (pdf/png)
 - identification exports
 - intermediate tables
 - 02_Run_002/
 - detail/
 - cohort exports (often in `histone_layouts/` or dataset-level export location)
 - histone_ratios.xls (often TSV text)

45.3 “Green flags” (quick signs of a healthy run)

- Each run has a `01_<raw_name>/detail/` folder with content.
- Cohort export exists and is non-empty.
- Anchor regions have non-zero **Area** in most samples.
- No unexpected duplication of run folders (e.g. `01_...` repeated due to reordering).

45.4 “Red flags” (stop and fix before analysis)

- `histone_layouts/` exists but per-run `detail/` is empty.
- Cohort export has headers but almost no data rows.
- Many samples have near-zero detected features.
- RT values are 0 or implausible (outside gradient).

46. QC packet template (what to save)

The QC packet is a small set of files you keep for every dataset. It should be sufficient to:

- prove the run is technically valid,
- compare datasets across time,
- support a Methods/Appendix description.

46.1 Minimum QC tables

Save as TSV/CSV:

1. `qc_feature_counts.tsv`

Columns:

- `Sample`
- `n_features_area`
- `n_features_ratio`
- `n_regions_detected` (optional)

2. `qc_intensity_summary.tsv`

Columns:

- `Sample`
- `median_log10_area`
- `iqr_log10_area`
- `min_log10_area` (optional)
- `max_log10_area` (optional)

3. `qc_anchor_rt.tsv`

Columns:

- `Sample`
- `Region`
- `RT_min` (or median RT)
- `Area` (or median area)

4. `qc_missingness.tsv`

Columns:

- `Sample`
- `frac_missing_area`
- `frac_missing_ratio`

46.2 Minimum QC plots

Save as PDF/PNG with stable names:

- `qc_area_boxplot.pdf`

Boxplot of `log10(Area)` per sample.

- `qc_pca_ratio.pdf`

PCA on a filtered ratio matrix (document filter rule).

- `qc_heatmap_marks.pdf` (optional but recommended)

Heatmap of selected hPTM summaries (or selected hPF ratios).

- `qc_anchor_rt.pdf`

RT scatter/line plot for 3–5 anchor regions across samples.

46.3 Naming rules

Use stable, sortable filenames:

- `qc_01_feature_counts.tsv`
- `qc_02_missingness.tsv`
- `qc_03_area_boxplot.pdf`
- `qc_04_pca_ratio.pdf`

Never overwrite QC without changing the run manifest or commit. If you rerun, write a new QC packet folder (e.g. `qc_v2/`).

47. Glossary (short)

- **Area:** integrated XIC signal for a feature (intensity proxy).
 - **Ratio:** within-peptide compositional abundance of a peptideform.
 - **RT(min):** retention time in minutes (as reported/exported).
 - **XIC:** extracted ion chromatogram (intensity vs RT for an m/z trace).
 - **hDP:** histone-derived peptide (region-level entity).
 - **hPF:** histone peptideform (specific modified form; often export rows).
 - **hPTM:** marginal PTM summary derived from hPF ratios.
 - **Layout:** curated region definitions and expected feature sets used by MATLAB.
 - **RT reference cache:** persisted RT anchors under `histone_layouts/` guiding extraction.
-

48. Quickstart checklist (copy/paste)

Use this checklist when you process a dataset.

1. Folder hygiene

- ☐ One homogeneous group per `raw_path`
- ☐ `histone_layouts/` fresh or reuse explicitly documented

2. Inputs

- ☐ MS1/MS2 files exist for every `raw_name`
- ☐ No basename collisions
- ☐ Conversion and extraction logs are saved

3. MATLAB run

- ☐ Per-run `detail/` folders created
- ☐ Cohort export created (`histone_ratios`)

4. Parsing

- ☐ Export parsed to long format
- ☐ Key uniqueness check passes
- ☐ Metric labels correct (`Ratio`, `Area`, `RT(min)`)

5. QC

- ☐ Feature counts per sample look plausible
- ☐ `log10(Area)` distributions comparable

- ☐Anchor RTs plausible and stable
- ☐Missingness within acceptable thresholds

6. Downstream

- ☐Mark rules validated (include/exclude)
- ☐hPTM summaries computed only after validation
- ☐Run manifest saved with commit hash + parameters

49. Next section to write

- “Common workflows”: public PXD reanalysis vs internal atlas processing.
- “Deployment notes”: Windows/WSL vs Linux, MATLAB path setup, external tool dependencies.
- A minimal troubleshooting appendix with “symptom → likely cause → check → fix”.

50. ProteomeXchange setup (R): 00_complete_pipeline_setup.R

This repository includes an R script that bootstraps a ProteomeXchange workspace: it inventories PXDs, writes inventory tables, and creates a standard folder scaffold under a single project root.

File:

- 00_complete_pipeline_setup.R

50.1 What it does (as written)

1. Loads/installs packages:

- Bioconductor: `rpx` (via `BiocManager::install("rpx")`)
- CRAN: `dplyr`, `tibble`, `readr`, `cli`, `crayon`

2. Uses a hard-coded project root:

- `ROOT_DIR <- "E:/EpiProfile_2.0_PLANTS"`
- Creates:
 - `E:/EpiProfile_2.0_PLANTS/PX_INVENTORY`
 - `E:/EpiProfile_2.0_PLANTS/PX_DATA`

3. Inventories a hard-coded list of PXDs:

- `pxd_vec <- c("PXD010102", "PXD046034", "PXD014739", "PXD046788", "PXD031991")`

4. For each PXD:

- calls `PXDataset(px_id)`
- calls `pxfiles(px, as.vector = FALSE)`
- classifies file types by extension:
 - RAW: `.raw`
 - Sciex: `.wiff`, `.wiff.scan`
 - mzML: `.mzml`, `.mzml.gz`

- ID formats: `.mzid`, `.mzid.gz`, `.mztab`, `.mztab.txt`

5. Writes inventory outputs to `PX_INVENTORY/` with a timestamp:

- `px_files_inventory_<timestamp>.tsv`
- `px_files_summary_<timestamp>.tsv`
- `px_projects_<timestamp>.tsv`
- `px_files_selected_<timestamp>.tsv` (filters to RAW/WIFF only, and sets `selected_for_epiprofile = TRUE`)

6. Creates the per-PXD folder scaffold under `PX_DATA/<PXD>/`:

- `raw/wiff`
- `raw/raw_thermo`
- `raw/other`
- `mzML/profile`
- `mzML/centroid`
- `MS1_MS2`
- `epiprofile_output`
- `metadata`
- `logs`
- `scripts`

It also writes a small `PX_DATA/<PXD>/README.md` describing these folders.

50.2 Important notes (based on the file content)

- Paths are Windows-style and hard-coded (`E:/...`). If you are not on that layout, you must change `ROOT_DIR`.
- The script contains a large download-related block (functions like `try_download_methods()`, `download_large_file()`, and a loop over `selected_files`) that, as written, references objects not introduced earlier in the file (e.g. `selected_files`, `download_dir`) and calls `progress_bar$new()` without loading a package that defines it. Treat that embedded download block as “draft code” unless you have already validated it locally.
- A separate downloader script exists (`01_download_raw.R`) and is the cleaner place to start for downloads.

51. Downloader (R): `01_download_raw.R`

File:

- `01_download_raw.R`

Purpose:

- Download the RAW/WIFF files selected in the most recent `px_files_selected_*.tsv` created by `00_complete_pipeline_setup.R`.

51.1 What it does (as written)

1. Uses:

- `ROOT_DIR <- "E:/EpiProfile_2.0_PLANTS"`
- `inventory_dir <- file.path(ROOT_DIR, "PX_INVENTORY")`
- `download_root <- file.path(ROOT_DIR, "PX_DATA")`

2. Automatically finds the most recent inventory selection table:

- scans `PX_INVENTORY/` for `px_files_selected_.*\.tsv`
- sorts and takes the last one
- reads it as `selected_files`

3. Downloads each selected file into:

- Thermo RAW: `PX_DATA/<PXD>/raw/raw_thermo/<filename>`
- Sciex WIFF: `PX_DATA/<PXD>/raw/wiff/<filename>`

4. Download logic (`download_with_retry()`):

- converts `ftp://...` URLs to `https://...` via `sub("^ftp://", "https://", url)`
- sets R timeout (default in function: `timeout_sec = 600`)
- uses `download.file(..., method = "libcurl")`
- deletes any partial output file before each attempt
- after each attempt, checks output file size and considers success if `> 5 MB` (the threshold is hard-coded)

5. Tracks status in the table:

- writes `status_download` and `download_message`
- saves incremental progress every 10 files:
 - `<px_files_selected_...>_DOWNLOADING.tsv`
- saves final:
 - `<px_files_selected_...>_DOWNLOADED.tsv`

51.2 Notes you should not miss

- The file ends with:
 - `setwd("E:/EpiProfile_2.0_PLANTS")`
 - `source("01_download_raw.R")` This will re-source the script from inside itself if executed as-is. In practice, those lines should be removed/ignored when running with `Rscript`, otherwise you risk recursion or confusing behavior.

52. Shiny converter (R): EpiProfile_PLANTS - Shiny Converter.R

File:

- `EpiProfile_PLANTS - Shiny Converter.R`

This is a Shiny app that wires together:

- WIFF/WIFFSCAN → mzML (via WSL + a bash script)
- mzML → MS1/MS2 (via PowerShell + a `.ps1` script)

52.1 Dependencies (as declared)

CRAN packages:

- `shiny`, `shinydashboard`, `shinyjs`, `fs`

The script installs missing packages on load (`install.packages(..., dependencies = TRUE)`).

52.2 Expected external scripts (hard-coded paths)

The app assumes a project structure where:

- `project_root = dirname(raw_dir)` or `dirname(mzml_dir)` (parent folder)
- Under that parent folder, it expects:
 - `scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh`
 - `scripts_pre_EpiProfile_PLANTS/02_extract_MS1_MS2.ps1`

If these are missing, the app returns an explicit error message (it checks `file_exists()`).

52.3 WIFF → mzML: how it runs it

Function:

- `run_msconvert(raw_dir, mzml_dir, log_file = NULL)`

Key behavior:

- checks for `.wiff` files in `raw_dir`
- converts Windows paths to WSL paths with `win_to_wsl()` (drive letter → `/mnt/<drive>/...`)
- calls:
 - `cmd <- "wsl"`
 - `args <- c("bash", <sh_script_wsl>, <raw_wsl>, <mzml_wsl>)`
- captures stdout/stderr and returns `success + message`

52.4 mzML → MS1/MS2: how it runs it

Function:

- `run_xtract_xml(mzml_dir, ms1ms2_dir, log_file = NULL)`

Key behavior:

- checks for `.mzML` files in `mzml_dir`
- calls:
 - `cmd <- "powershell.exe"`
 - `args <- c("-NoProfile", "-ExecutionPolicy", "Bypass", "-File", <ps_script>, "-InputDir", <mzml_dir>, "-OutputDir", <ms1ms2_dir>)`
- captures stdout/stderr and returns `success + message`

The file ends with:

- `shinyApp(ui = ui, server = server)`

So the app runs directly when you source the file in an interactive R session.

53. mzML QC helper (bash): `QC_MZML_AUTO.sh`

File:

- `QC_MZML_AUTO.sh`

What is present in this file:

- detects system resources:
 - memory (GB) via `free -g`
 - CPU cores via `nproc`
- supports modes:
 - `auto` (default)
 - `piloto`
 - `balanceado`
 - `produccion`
- in `auto`, selects the mode based on detected memory:
 - `< 8 GB` → `piloto`
 - `> 64 GB` → `produccion`
 - otherwise → `balanceado`
- sets two variables:
 - `PARALELISMO`
 - `MUESTRAS_LIMITE`
- prints the chosen strategy

The file explicitly ends the “real processing” section with a placeholder comment:

- `# ... (el resto del script similar a los anteriores)`

So, as committed here, it is a resource-based mode selector plus a stub.

54. Interactive analysis prototype (R/Shiny): `Análisis de EpiProfile - Histonas Arabidopsis.R`

File:

- `Análisis de EpiProfile - Histonas Arabidopsis.R`

This is a Shiny app focused on interactive visualization of histone PTM outputs.

54.1 What is clearly implemented

- Loads a large stack of packages (`plotly`, `DT`, `pheatmap`, `FactoMineR`, `ComplexHeatmap`, etc.).
- Implements a parser:
 - `parse_histone_ratios_file(file_path)`
 - reads the file line-by-line (`readLines`)
 - treats line 1 as the sample header (with comma-separated tokens) and line 2 as the “data type line”

- extracts sample names by splitting on tabs, then removing a leading numeric prefix before the comma
- computes a simple `Sample_Group` as “letters from the last hyphen-separated token”
- parses modification rows into a wide format where each sample produces:
 - `<sample>_Ratio`, `<sample>_Area`, `<sample>_RT`
- builds:
 - `ratios_matrix` from `_Ratio` columns
 - `areas_matrix` from `_Area` columns

54.2 What is not implemented (in this file)

- The “unmod variant” parsing branch is present but explicitly incomplete:
 - it contains a comment: “Implementación similar omitida por brevedad”
 - and does not construct the unmod table.

The file ends with:

- `shinyApp(ui = ui, server = server)`

So it is runnable as a Shiny app, but the parsing logic is specialized and includes incomplete parts.

55. Next section (grounded): “How these scripts map to the repository pipeline”

55. How these scripts map to the repository pipeline

This repository has “pre-EpiProfile” helper scripts (download + conversion + extraction) and then the MATLAB run that creates `histone_layouts/` and cohort exports.

This section is only about mapping the *existing scripts* to the pipeline and the *paths they actually use* in code.

55.1 Script → role (what it is for)

A) `00_complete_pipeline_setup.R`

- Purpose: inventory selected ProteomeXchange projects (PXD) and create a per-PXD folder tree under a single project root. It creates (among others) `raw/`, `mzML/`, `MS1_MS2/`, `epiprofile_output/`, `metadata/`, `logs/`, `scripts/`:`contentReference[oaicite:0]{index=0}`

B) `01_download_raw.R`

- Purpose: find the latest `px_files_selected_*.tsv` in `PX_INVENTORY/`, then download files into `PX_DATA/<PXD>/raw/raw_thermo` or `PX_DATA/<PXD>/raw/wiff` depending on file type:`contentReference[oaicite:1]{index=1}`:`contentReference[oaicite:2]{index=2}`

C) `EpiProfile_PLANTS - Shiny Converter.R`

- Purpose: a Shiny “front-end” to run two external steps:

1. WIFF/WIFFSCAN → mzML via a WSL bash call to
`scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh`
2. mzML → MS1/MS2 via a Windows PowerShell call to
`scripts_pre_EpiProfile_PLANTS/02_extract_MS1_MS2.ps1:contentReference[oaicite:3]
{index=3}:contentReference[oaicite:4]{index=4}:contentReference[oaicite:5]{index=5}`

D) QC_MZML_AUTO.sh

- Purpose: a Linux/WSL bash wrapper that auto-selects a “mode” (pilot/balanced/production) based on RAM/cores and sets `PARALELISMO` and an optional sample limit. The script header explicitly documents usage as `./QC_MZML_AUTO.sh [modo] [ruta_datos]:contentReference[oaicite:6]
{index=6}:contentReference[oaicite:7]{index=7}`

55.2 The “two roots” you will see in practice

There are two different “roots” hard-coded / defaulted in these scripts:

1. ProteomeXchange staging root (R scripts)

- `ROOT_DIR <- "E:/EpiProfile_2.0_PLANTS"` in `01_download_raw.R`. From there:
 - `inventory_dir <- E:/EpiProfile_2.0_PLANTS/PX_INVENTORY`
 - `download_root <- E:/EpiProfile_2.0_PLANTS/PX_DATA:contentReference[oaicite:8]
{index=8}`

2. Local conversion root (Shiny Converter defaults)

- The Shiny UI defaults to a *Windows* project folder, e.g.:
 - `project_root = "C:/Users/geope/Desktop/EpiProfile_PLANTS_ATLAS"`
 - `raw_wiff_dir = "C:/Users/geope/Desktop/EpiProfile_PLANTS_ATLAS/raw_wiff"`
 - `mzml_dir = "C:/Users/geope/Desktop/EpiProfile_PLANTS_ATLAS/mzML"`
 - `ms1ms2_dir = "C:/Users/geope/Desktop/EpiProfile_PLANTS_ATLAS/MS1_MS2":contentReference[oaicite:9]
{index=9}`

Important: these are *two different workflows*. Decide which one you are using per dataset, and keep it consistent.

55.3 Verified per-PXD directory layout (created by `00_complete_pipeline_setup.R`)

When `00_complete_pipeline_setup.R` creates the per-PXD structure, it uses the following subdirectories (verbatim list in the script):contentReference[oaicite:10]{index=10}

- `raw/wiff`
- `raw/raw_thermo`
- `raw/other`
- `mzML/profile`
- `mzML/centroid`
- `MS1_MS2`
- `epiprofile_output`
- `metadata`

- logs
- scripts

So, for a given PXD (example pattern), the *verified* path shapes are:

- E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/raw/wiff/
- E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/raw/raw_thermo/
- E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/mzML/profile/
- E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/mzML/centroid/
- E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/MS1_MS2/
- E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/epiprofile_output/

These are “guessed” only in the sense that <PXD> is a placeholder token; the directory *names* are not guessed (they are in the script).

55.4 Download step (where files really land)

01_download_raw.R decides the destination subfolder from flags in the selection table:

- Thermo RAW → raw/raw_thermo
- Sciex WIFF → raw/wiff:contentReference[oaicite:11]{index=11}

It then writes to:

- dest_dir <- file.path(download_root, fi\$px_id, subdir)
- dest_file <- file.path(dest_dir, fi\$name):contentReference[oaicite:12]{index=12}

So the concrete pattern is:

- E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/raw/raw_thermo/<filename.raw>
 - E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/raw/wiff/<filename.wiff>:contentReference[oaicite:13]{index=13}
-

55.5 Conversion + extraction step (what the Shiny Converter expects)

The Shiny Converter’s key assumption is this:

- It deduces project_root <- dirname(raw_dir) (so your RAW folder must be directly under the project root):contentReference[oaicite:14]{index=14}
- It requires that two helper scripts exist under:
 - <project_root>/scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh
 - <project_root>/scripts_pre_EpiProfile_PLANTS/02_extract_MS1_MS2.ps1:contentReference[oaicite:15]{index=15}:contentReference[oaicite:16]{index=16}

And it runs them as:

1. WIFF → mzML

- Uses WSL and calls: wsl bash <script> <raw_wsl> <mzml_wsl>:contentReference[oaicite:17]{index=17}

2. mzML → MS1/MS2

- Uses PowerShell and calls the `.ps1` with `-InputDir <mzml_dir>` and `-OutputDir <ms1ms2_dir>:contentReference[oaicite:18]{index=18}`

So the Shiny workflow is “self-contained” inside one Windows project folder with these children:

- `raw_wiff/`
 - `mzML/`
 - `MS1_MS2/`
 - `scripts_pre_EpiProfile_PLANTS/` (must contain the two scripts above):`contentReference[oaicite:19]{index=19}:contentReference[oaicite:20]{index=20}`
-

55.6 Where `QC_MZML_AUTO.sh` fits (and what it really does)

`QC_MZML_AUTO.sh` is a resource-aware wrapper. What is explicitly implemented in the snippet you have:

- CLI usage: `./QC_MZML_AUTO.sh [modo] [ruta_datos]:contentReference[oaicite:21]{index=21}`
- Reads RAM (GB) and CPU cores, then chooses a mode if `modo=auto`:`contentReference[oaicite:22]{index=22}`
- Sets:
 - `PARALELISMO=1` and `MUESTRAS_LIMITE=3` for “piloto”
 - `PARALELISMO=CPU/2` for “balanceado”
 - `PARALELISMO=CPU` for “produccion”:`contentReference[oaicite:23]{index=23}`

The script ends with a placeholder comment indicating the “rest of the processing” is not included in that file fragment.:`contentReference[oaicite:24]{index=24}`

Practical mapping: treat it as optional QC glue you can run around your mzML set (especially on WSL/Linux), but do not assume it performs full QC unless you have completed the “rest of the script”.

55.7 Hand-off point to MATLAB (pipeline intent)

Once you have a clean `MS1_MS2/` folder for a dataset, that is the hand-off point to the MATLAB EpiProfile run you documented earlier (the run that creates `histone_layouts/` under the chosen `raw_path`).

In other words, the outputs of the Shiny Converter’s step (or your own extractor) should end up in *one* MS1/MS2 folder, and that folder becomes `raw_path`.

A conservative, repository-consistent choice (using the per-PXD structure created by `00_complete_pipeline_setup.R`) is:

- `raw_path = E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/MS1_MS2:contentReference[oaicite:25]{index=25}`

(That path exists because the script creates `MS1_MS2/` for every PXD it sets up.)

55.8 Minimal mental model (one screen)

ProteomeXchange path:

- `00_complete_pipeline_setup.R` → creates `E:/EpiProfile_2.0_PLANTS/PX_DATA/<PXD>/...` and inventory tables.:`contentReference[oaicite:26]{index=26}`

- `01_download_raw.R` → fills `.../raw/wiff` or `.../raw/raw_thermo`:`contentReference[oaicite:27]{index=27}`
- (Then you convert/extract into `.../mzML/...` and finally `.../MS1_MS2/...`)
- MATLAB EpiProfile run consumes `.../MS1_MS2` and creates `histone_layouts/` there (documented earlier in this manual).

Local WIFF path:

- Shiny Converter expects `<project_root>/raw_wiff` → `<project_root>/mzML` → `<project_root>/MS1_MS2`, plus `<project_root>/scripts_pre_EpiProfile_PLANTS/*.sh/*.ps1`:`contentReference[oaicite:28]{index=28}`:`contentReference[oaicite:29]{index=29}`
- MATLAB EpiProfile run consumes `<project_root>/MS1_MS2`.

55.9 What to document next (still grounded)

56. Workspace alignment: PX_DATA scaffold vs Shiny Converter expectations

This is the first place where “folder conventions” can quietly break your pipeline.

56.1 What the Shiny Converter assumes (as written)

The converter does not really use the UI field `project_root` to locate scripts. Instead:

- For WIFF → mzML, it sets `project_root <- dirname(raw_dir)`
- For mzML → MS1/MS2, it sets `project_root <- dirname(mzml_dir)`

Then it expects BOTH helper scripts to exist at:

- `<project_root>/scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh`
- `<project_root>/scripts_pre_EpiProfile_PLANTS/02_extract_MS1_MS2.ps1`

Implication: `raw_dir` and `mzml_dir` should be siblings under the same parent directory (the “real `project_root`”), exactly like the default UI example:

- `<root>/raw_wiff/`
- `<root>/mzML/`
- `<root>/MS1_MS2/`
- `<root>/scripts_pre_EpiProfile_PLANTS/`

If you point `raw_dir` into a deeper tree (e.g. `.../raw/wiff/`), the script lookup moves with it.

56.2 What the PX_DATA scaffold creates (as written)

The setup script creates a PXD tree like:

- `<ROOT_DIR>/PX_DATA/<PXD>/raw/wiff/`
- `<ROOT_DIR>/PX_DATA/<PXD>/raw/raw_thermo/`
- `<ROOT_DIR>/PX_DATA/<PXD>/mzML/profile/`
- `<ROOT_DIR>/PX_DATA/<PXD>/mzML/centroid/`

- `<ROOT_DIR>/PX_DATA/<PXD>/MS1_MS2/`
- `<ROOT_DIR>/PX_DATA/<PXD>/scripts/` (note: **scripts**, not `scripts_pre_EpiProfile_PLANTS`)

This is a valid scaffold, but it does not match the Shiny Converter’s “siblings under one root” assumption unless you adapt paths or adapt code.

56.3 Two consistent ways to run (pick one per dataset)

Option A (Shiny-style workspace; simplest)

- Use one dataset root folder that follows the Shiny defaults:
 - `<dataset_root>/raw_wiff/`
 - `<dataset_root>/mzML/`
 - `<dataset_root>/MS1_MS2/`
 - `<dataset_root>/scripts_pre_EpiProfile_PLANTS/`
- Put `.wiff` and `.wiff.scan` in `raw_wiff/`.
- Convert into `mzML/`.
- Extract into `MS1_MS2/`.
- Use `raw_path = <dataset_root>/MS1_MS2` for MATLAB.

Option B (PX_DATA scaffold; keep ProteomeXchange structure)

- Use the per-PXD tree under `PX_DATA/<PXD>/`.
- Do **not** assume the Shiny Converter will “just work” with deep folders unless you do one of:
 1. Create sibling convenience folders at `<PXD>/` that match Shiny expectations (recommended if you want to keep Shiny):
 - Create `<PXD>/raw_wiff/` that points to `<PXD>/raw/wiff/` (symlink or copy)
 - Create `<PXD>/mzML/` that points to either `<PXD>/mzML/profile/` or `<PXD>/mzML/centroid/` (symlink or copy)
 - Keep `<PXD>/MS1_MS2/` as-is (already matches)
 - Add `<PXD>/scripts_pre_EpiProfile_PLANTS/` with the two helper scripts
 2. Or modify the Shiny Converter to use `input$project_root` instead of `dirname(raw_dir) / dirname(mzml_dir)` (code change; keep it versioned)

If you do neither, the converter will search for helper scripts in the wrong place.

57. Where to put the helper scripts (converter requirements)

The Shiny Converter requires exactly these filenames:

- `scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh`
- `scripts_pre_EpiProfile_PLANTS/02_extract_MS1_MS2.ps1`

And it will only find them if they are located under the parent folder of:

- your chosen `raw_wiff_dir` (for conversion), and
- your chosen `mzml_dir` (for extraction)

Practical rule:

- ensure `dirname(raw_wiff_dir) == dirname(mzml_dir) == dirname(ms1ms2_dir)`

- and ensure that shared parent contains `scripts_pre_EpiProfile_PLANTS/`
-

58. Logging behavior (what you actually get today)

58.1 Downloader (`01_download_raw.R`)

- Writes progress tables next to the selection list in `PX_INVENTORY/`:
 - `..._DOWNLOADING.tsv`
 - `..._DOWNLOADED.tsv`
- Logs are mostly console prints unless you redirect output when running (e.g. `sink()` or shell redirection).

58.2 Shiny Converter (`EpiProfile_PLANTS - Shiny Converter.R`)

- The converter functions accept `log_file`, but the server calls them with `log_file = NULL`.
- Therefore, by default:
 - output is shown in the Shiny UI text boxes,
 - no log file is written to disk unless you modify the app to pass a path.

If you want persistent logs without changing much:

- add a `logs/` folder under your dataset root
- pass `log_file = file.path(<root>, "logs", "msconvert.log")` and similar for extraction.

58.3 mzML QC wrapper (`QC_MZML_AUTO.sh`)

- As committed here, it selects a mode and sets variables.
 - It does not implement a full QC pass in the shown content (it ends with a placeholder comment). Treat it as “mode selection + stub” unless you extend it.
-

59. Pipeline hand-off: what MATLAB expects from pre-steps

By the time you enter MATLAB, you should have:

- one folder (`raw_path`) containing extracted MS1/MS2 files for all runs in the batch
- a clean mapping of basenames (`raw_names`) to MS1/MS2 pairs

Most downstream problems that look like “MATLAB issues” are actually:

- wrong pairing (MS1/MS2 mismatch),
- missing files for a basename,
- reused `histone_layouts/` with incompatible RT reference state.

Operational rule:

- treat `raw_path` as the atomic dataset unit (homogeneous LC method + instrument + extraction method).

60. What we can document next without guessing

The next grounded step is to document the MATLAB “entrypoint(s)” and the cohort export(s) *from your repository code*, by locating:

1. the `.m` file(s) that call `DrawISOProfile*` (or equivalent),
2. the `.m` file(s) that write the combined cohort export (`histone_ratios...`),
3. the exact output location rules for that export.

To do that deterministically in your checkout, use one of:

Linux/WSL:

- `find . -name "DrawISOProfile*.m" -o -name "*.m" | wc -l`
- `grep -R --line-number "DrawISOProfile" .`
- `grep -R --line-number "histone_ratios" .`
- `grep -R --line-number "histone_layouts" .`

Windows PowerShell:

- `Get-ChildItem -Recurse -Filter *.m | Select-String -Pattern "DrawISOProfile" -List`
- `Get-ChildItem -Recurse -Filter *.m | Select-String -Pattern "histone_ratios" -List`

Once you have those filenames/paths, the manual can pin:

- the exact MATLAB command you run,
- the exact export filenames,
- the exact “output contract” (row keys, blocks, metrics).

61. Deployment notes (Windows + WSL) — what this repo actually runs

This repo already contains two execution styles:

A) R scripts for ProteomeXchange inventory + download (Windows paths by default)

B) A Shiny “Converter” app that calls:

- `wsl bash ...` for WIFF → mzML
- `powershell.exe ...` for mzML → MS1/MS2

Everything below is grounded in what the scripts literally execute today.

61.1 Minimum runtime assumptions (because the code calls them)

For the Shiny Converter to work as written, your machine must have:

- `wsl` available on PATH (the app calls `system2("wsl", ...)`)

- `powershell.exe` available (the app calls `system2("powershell.exe", ...)`)

The Converter does not call `msconvert` directly; it calls a bash script:

- `<project_root>/scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh`

So `msconvert` (and possibly Docker) is a dependency of *that* script, not of the R app itself.

61.2 “Project root” rule (do not miss this)

The Shiny Converter does **not** use the UI variable `project_root` to locate scripts.

It computes:

- `project_root <- dirname(raw_dir)` for WIFF → mzML
- `project_root <- dirname(mzml_dir)` for mzML → MS1/MS2

Then it looks for scripts under:

- `<project_root>/scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh`
- `<project_root>/scripts_pre_EpiProfile_PLANTS/02_extract_MS1_MS2.ps1`

If your `raw_dir` is nested (e.g. `.../<PXD>/raw/wiff`), the converter will search for scripts in `.../<PXD>/raw/scripts_pre_EpiProfile_PLANTS`, which is almost certainly wrong.

Operational rule:

- make `raw_dir`, `mzml_dir`, and `ms1ms2_dir` siblings under the same dataset root folder.

62. Running the Shiny Converter (verified behavior)

File:

- `EpiProfile_PLANTS - Shiny Converter.R`

This file ends with `shinyApp(ui = ui, server = server)`, so sourcing it will start the app.

62.1 What you must provide (inputs in UI)

- A “raw WIFF folder” containing at least one `.wiff` file
- A “mzML output folder”
- A “MS1/MS2 output folder”

62.2 What the app will do (as coded)

Step 1: WIFF → mzML

- checks `raw_dir` exists
- creates `mzml_dir` if needed
- checks there is at least one `.wiff` file in `raw_dir`
- checks existence of:

- `<dirname(raw_dir)>/scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh`
- converts Windows paths to WSL paths (`C:/... → /mnt/c/...`)
- runs:
 - `wsl bash <script> <raw_dir_wsl> <mzml_dir_wsl>`

Step 2: mzML → MS1/MS2

- checks `mzml_dir` exists
- creates `ms1ms2_dir` if needed
- checks there is at least one `.mzML` file in `mzml_dir`
- checks existence of:
 - `<dirname(mzml_dir)>/scripts_pre_EpiProfile_PLANTS/02_extract_MS1_MS2.ps1`
- runs:
 - `powershell.exe -NoProfile -ExecutionPolicy Bypass -File <ps_script> -InputDir <mzml_dir> -OutputDir <ms1ms2_dir>`

62.3 Where logs go

By default, the app does not write a log file to disk (`log_file = NULL` in its internal calls). The messages are returned and printed in the app UI.

If you need persistent logs, you must edit the app so it passes a `log_file` path to:

- `run_msconvert(..., log_file=...)`
- `run_xtract_xml(..., log_file=...)`

63. Running the ProteomeXchange staging scripts (verified behavior)

These scripts assume a Windows root, hard-coded by default:

- `ROOT_DIR <- "E:/EpiProfile_2.0_PLANTS"`

63.1 00_complete_pipeline_setup.R (inventory + scaffold)

What it creates:

- `<ROOT_DIR>/PX_INVENTORY/`
- `<ROOT_DIR>/PX_DATA/`

For each PXD in its hard-coded vector, it creates:

- `PX_DATA/<PXD>/raw/wiff`
- `PX_DATA/<PXD>/raw/raw_thermo`
- `PX_DATA/<PXD>/raw/other`
- `PX_DATA/<PXD>/mzML/profile`
- `PX_DATA/<PXD>/mzML/centroid`
- `PX_DATA/<PXD>/MS1_MS2`
- `PX_DATA/<PXD>/epiprofile_output`

- `PX_DATA/<PXD>/metadata`
- `PX_DATA/<PXD>/logs`
- `PX_DATA/<PXD>/scripts`

It also writes inventory tables to `PX_INVENTORY/` with timestamps.

63.2 `01_download_raw.R` (download RAW/WIFF)

What it does:

- finds the most recent `px_files_selected_*.tsv` in `PX_INVENTORY/`
- downloads each selected file into:
 - `PX_DATA/<PXD>/raw/raw_thermo/` for Thermo RAW
 - `PX_DATA/<PXD>/raw/wiff/` for Sciex WIFF

Download behavior to know:

- it retries downloads
- it deletes partial files before re-attempt
- it considers a file “OK” only if it is larger than a hard-coded threshold (5 MB in the script)

Important footnote:

- the file ends with `setwd(...)` and `source("01_download_raw.R")`. If you are running it with `Rscript`, you should remove/ignore those lines to avoid re-sourcing itself.

64. Bridging the two worlds: PX_DATA scaffold vs Shiny Converter

These are compatible, but not automatically.

64.1 Why they don't align by default

`PX_DATA` puts WIFF files in:

- `<PXD>/raw/wiff/`

Shiny Converter wants WIFF files in a folder whose **parent** also contains:

- `scripts_pre_EpiProfile_PLANTS/`

So if you point Shiny Converter at `<PXD>/raw/wiff/`, it will look for scripts at:

- `<PXD>/raw/scripts_pre_EpiProfile_PLANTS/...` (wrong for most setups)

64.2 The conservative bridge (no code changes)

If you want to keep using the Shiny Converter without editing it:

Inside `PX_DATA/<PXD>/`, create a dataset root folder that matches the Shiny layout, for example:

- `PX_DATA/<PXD>/raw_wiff/` (copy or symlink from `raw/wiff/`)

- `PX_DATA/<PXD>/mzML/` (point to `mzML/profile/` or use a new folder)
- `PX_DATA/<PXD>/MS1_MS2/` (already exists)
- `PX_DATA/<PXD>/scripts_pre_EpiProfile_PLANTS/` (place the 2 helper scripts here)

Then you can safely select:

- `raw_dir = PX_DATA/<PXD>/raw_wiff`
- `mzml_dir = PX_DATA/<PXD>/mzML`
- `ms1ms2_dir = PX_DATA/<PXD>/MS1_MS2`

and the Converter will resolve scripts under the correct parent folder.

65. Output export formats: detect first, parse second

In this project, files named `*.xls` may be plain TSV text.

Separately, there are (at least) two common structural patterns for the export:

65.1 Pattern A — “classic EpiProfile” row layout (what the analysis Shiny app assumes)

- line 1: sample header line (sample names repeated per block)
- line 2: a “data types” line (the Shiny parser reads it but does not use it)
- peptide header lines appear as a single string like:
 - `TKQTAR(H3_3_8)`
- modification lines contain a modification name then numeric blocks:
 - Ratio block for all samples
 - separator column(s)
 - Area block for all samples
 - separator column(s)
 - RT block for all samples

This is exactly what `parse_histone_ratios_file()` in:

- `Análisis de EpiProfile - Histonas Arabidopsis.R` tries to parse.

65.2 Pattern B — “2-row header table” (what the conservative pandas template assumes)

- row 1: sample names (wide)
- row 2: metric labels (Ratio/Area/RT(min))
- row 3+: data rows

This is the format used by the conservative parsing template you already wrote (the `h1/h2` approach).

65.3 Detection rule (simple)

Before you parse, read the first 2 rows as raw strings.

- If row 2 contains repeating metric labels like `Ratio`, `Area`, `RT` aligned across columns → Pattern B is likely.
- If row 1 looks like sample names repeated 3 times and there are blank columns between blocks → Pattern A is likely.

Do not force one parser on the other format.

66. Known limitations of the existing Shiny analysis parser (do not rely on it blindly)

File:

- `Análisis de EpiProfile - Histonas Arabidopsis.R`

The `parse_histone_ratios_file()` implementation has two hard limitations as written:

1. It truncates samples:

- after building `sample_names <- unique(sample_names)`, it explicitly says:
 - “Tomar solo las primeras 4 muestras únicas” So it will silently ignore samples beyond the first four unique names.

2. It does not implement unmodified-variant parsing:

- the `unmod(...)` branch is a stub (“omitida por brevedad”).

Meaning:

- treat this Shiny app as an *interactive prototype*, not as the canonical parser for cohort exports.
-

67. Canonical sequence notes (internal project files)

These files exist as project assets and are useful for microvariant reasoning:

- `H3_AT_MP_CR.txt` (FASTA-like sequences for H3 across selected taxa)
- `H4_AT_MP_CR.txt` (FASTA-like sequences for H4 across selected taxa)
- `canonical_AT_MP_CR` (a mixed markdown/narrative note summarizing motif positions and variant cases)

Use them for:

- verifying which short peptides/motifs exist in which isoforms
- documenting which “unmod variants” should be expected in layouts

Do not treat `canonical_AT_MP_CR` as a machine-readable table; it is a human note (tables + narrative).

68. Next sections to write (still “no invention”)

To continue grounding the manual, the next step is to extract from *your actual repo checkout*:

A) The MATLAB entrypoint(s)

- which `.m` file you run to start a dataset batch
- which function builds `raw_names`
- which function writes the cohort export

B) The exact cohort export filename(s) and locations

- the file name(s)
- where they are written relative to `raw_path`
- whether they are Pattern A or Pattern B (Section 65)

Once those are pinned, the manual can include:

- the exact MATLAB command-line invocation
- a formal output schema (required columns/blocks/invariants)
- a “safe RT reference” section tied to the exact functions in your build

69. Minimal reproducible run recipes (what you can do today)

This section gives “end-to-end” recipes using only what is present in this repo snapshot. It does not assume hidden MATLAB wrappers or extra helper scripts beyond what the Shiny Converter explicitly calls.

69.1 Recipe A — ProteomeXchange staging (inventory → download)

Files:

- `00_complete_pipeline_setup.R`
- `01_download_raw.R`

What is safe and deterministic:

1. Edit `ROOT_DIR` in both scripts (they default to `E:/EpiProfile_2.0_PLANTS`).
2. Run the inventory+scaffold step:
 - `Rscript 00_complete_pipeline_setup.R`

Outputs you should expect:

- `ROOT_DIR/PX_INVENTORY/px_files_inventory_<timestamp>.tsv`
- `ROOT_DIR/PX_INVENTORY/px_files_summary_<timestamp>.tsv`
- `ROOT_DIR/PX_INVENTORY/px_projects_<timestamp>.tsv`
- `ROOT_DIR/PX_INVENTORY/px_files_selected_<timestamp>.tsv`
- Per-PXD folders under `ROOT_DIR/PX_DATA/<PXD>/...` (raw/mzML/MS1_MS2/etc.)

Important caveat (verified in the file):

- `00_complete_pipeline_setup.R` contains a “download” block that references `selected_files` and `download_dir` even though it builds `px_files_selected` and `data_dir` earlier. As written, that block is not self-consistent and will error if executed unchanged.
- Practical implication: treat `00_complete_pipeline_setup.R` as “inventory + scaffold + selection tables”. Use `01_download_raw.R` for actual downloads.

3. Run the downloader:

- `Rscript 01_download_raw.R`

Important caveat (verified in the file):

- `01_download_raw.R` ends with:
 - `setwd("E:/EpiProfile_2.0_PLANTS")`
 - `source("01_download_raw.R")` If you execute with `Rscript`, remove/comment those lines first (otherwise it re-sources itself).

Outputs you should expect:

- Downloads in:
 - `ROOT_DIR/PX_DATA/<PXD>/raw/raw_thermo/` (Thermo RAW)
 - `ROOT_DIR/PX_DATA/<PXD>/raw/wiff/` (Sciex WIFF)
- Progress tables written next to the selection file:
 - `px_files_selected_<timestamp>_DOWNLOADING.tsv`
 - `px_files_selected_<timestamp>_DOWNLOADED.tsv`

69.2 Recipe B — Local conversion (WIFF → mzML → MS1/MS2) via Shiny

File:

- `EpiProfile_PLANTS - Shiny Converter.R`

This is a Shiny app that runs two external scripts:

- `<dataset_root>/scripts_pre_EpiProfile_PLANTS/01_convert_wiff_to_mzML.sh` called as: `ws1 bash <script> <raw_dir_ws1> <mzml_dir_ws1>`
- `<dataset_root>/scripts_pre_EpiProfile_PLANTS/02_extract_MS1_MS2.ps1` called as: `powershell.exe ... -File <ps1> -InputDir <mzml_dir> -OutputDir <ms1ms2_dir>`

Minimum folder layout to satisfy how the app resolves paths:

- `<dataset_root>/raw_wiff/`
- `<dataset_root>/mzML/`
- `<dataset_root>/MS1_MS2/`
- `<dataset_root>/scripts_pre_EpiProfile_PLANTS/` (must contain the two scripts above)

How to run:

- In an interactive R session (RStudio is fine):
 - `source("EpiProfile_PLANTS - Shiny Converter.R")`

Operational rule (verified from code behavior):

- The app does not use the UI field `project_root` to locate helper scripts.
 - It sets `project_root <- dirname(raw_dir)` and `project_root <- dirname(mzml_dir)`.
 - So `raw_dir`, `mzml_dir`, and `ms1ms2_dir` must be siblings under the same parent for script discovery to work reliably.
-

70. “What to edit first” in each script (verified knobs)

70.1 00_complete_pipeline_setup.R

Edit these at the top of section “CONFIGURACIÓN DE PARÁMETROS”:

- `pxd_vec <- c("PXD010102", "PXD046034", "PXD014739", "PXD046788", "PXD031991")`
- `ROOT_DIR <- "E:/EpiProfile_2.0_PLANTS"`

Everything downstream (inventory_dir, data_dir, output paths) derives from `ROOT_DIR`.

70.2 01_download_raw.R

Edit:

- `ROOT_DIR <- "E:/EpiProfile_2.0_PLANTS"`

The script auto-selects the most recent:

- `PX_INVENTORY/px_files_selected_*.tsv`

So you normally do not need to set a filename manually.

Also note:

- The downloader force-rewrites `ftp://...` URLs to `https://....`
- It considers a download “plausible” only if file size exceeds 5 MB (hard-coded threshold).

70.3 EpiProfile_PLANTS – Shiny Converter.R

If you want better defaults in UI, edit the `value = ...` fields for:

- `project_root`
- `raw_wiff_dir`
- `mzml_dir`
- `ms1ms2_dir`

But remember:

- these are UI defaults only
- actual script discovery depends on `dirname(raw_wiff_dir)` and `dirname(mzml_dir)`

71. Known “footguns” in this repo snapshot (verified from code)

71.1 00_complete_pipeline_setup.R: inconsistent download block

The script builds `px_files_selected` and writes `px_files_selected_<timestamp>.tsv`, then later contains code that references:

- `selected_files` (not defined in the earlier inventory/selection section)
- `download_dir` (not defined in the earlier section)
- `progress_bar$new(...)` (package not loaded in the “required_packages” list)

This is why, in practice, `00_complete_pipeline_setup.R` should be treated as:

- inventory + selection tables + scaffold and downloads should be handled by:
- `01_download_raw.R`

71.2 `01_download_raw.R`: self-sourcing tail

The last two lines:

- `setwd("E:/EpiProfile_2.0_PLANTS")`
- `source("01_download_raw.R")`

are not safe in a normal `Rscript` workflow. Remove or comment them before running non-interactively.

71.3 `Análisis de EpiProfile - Histonas Arabidopsis.R`: not a canonical parser

Its `parse_histone_ratios_file()` function explicitly:

- assumes a very specific “classic EpiProfile” row layout
- includes hard-coded logic for “4 samples” in comments and indexing
- contains an explicit note that `unmod(...)` parsing is omitted (“omitida por brevedad”)

Treat this as an interactive prototype, not as the authoritative export contract.

72. Verified interface contracts between the Shiny Converter and external scripts

The converter is strict about how it calls the helper scripts. This is important because you can debug “wrong outputs” just by checking the called arguments.

72.1 `01_convert_wiff_to_mzML.sh`

The app calls (via WSL):

- `ws1 bash <script> <raw_dir_wsl> <mzml_dir_wsl>`

So your bash script must accept exactly:

- argument 1: input WIFF folder
- argument 2: output mzML folder

72.2 `02_extract_MS1_MS2.ps1`

The app calls:

- `powershell.exe -NoProfile -ExecutionPolicy Bypass -File <ps1> -InputDir <mzml_dir> -OutputDir <ms1ms2_dir>`

So your PowerShell script must accept exactly:

- `-InputDir` and a directory path
- `-OutputDir` and a directory path

If your script uses different flag names, the converter will run but do nothing useful (or fail).

73. Confirming “ready for MATLAB” state (before you open MATLAB)

This is the minimum pre-condition for the MATLAB part of the pipeline (independent of how you got here):

A) `MS1_MS2/` contains paired MS1/MS2 files per run

You should be able to visually confirm that for each run you have:

- one MS1-like file
- one MS2-like file sharing the same basename.

B) `raw_names` will match the basenames

If you build `raw_names` automatically, verify that each name has both MS1 and MS2 present.

C) Folder hygiene

If you will reuse the same `raw_path` folder, decide explicitly:

- keep or delete `histone_layouts/` before running MATLAB (RT cache implications)
-

74. Next sections to write (still grounded, but requires repo-wide search)

75. Canonical peptide motifs (verified) for Arabidopsis / Marchantia / Chlamydomonas

This repository includes curated FASTA-like sequence bundles that are used as a “sanity anchor” when you claim a layout is portable across species:

- `H3_AT_MP_CR.txt`
- `H4_AT_MP_CR.txt`
- `canonical_AT_MP_CR` (human-readable notes + small motif tables)

The point is simple: before you reuse a layout (same peptide sequence, same PTM indexing assumptions), confirm the motif exists in the target species/isoform exactly as expected.

75.1 H3 N-terminus: TKQTAR and the K9/K14 peptide (verified positions)

From `H3_AT_MP_CR.txt`, all Arabidopsis H3 canonical isoforms listed contain **TKQTAR** starting at position 4 (1-based, counting the initial Met as position 1).

The K9/K14 peptide is normally **KSTGGKAPR** starting at position 10, but there is one important exception:

Arabidopsis entry	Symbols (as in FASTA header)	H3 class (by Symbols/description)	K9/K14 motif at pos 10
AT1G09200.1	H3.1, HTR2	H3.1	KSTGGKAPR
AT3G27360.1	H3.1, HTR3	H3.1	KSTGGKAPR
AT4G40040.1	H3.3, HTR5	H3.3	KSTGGKAPR
AT5G10390.1	H3.1, HTR13	H3.1	KSTGGKAPR
AT5G10400.1	H3.1, HTR9	H3.1	KSTGGKAPR
AT5G10980.1	H3.3, HTR8	H3.3	KSTGGKAPR
AT5G65360.1	H3.1, HTR1	H3.1	KSTGGKAPR
AT1G19890.1	ATMGH3, HTR10, MGH3	gamete-specific	KSTGGKGPR (variant; not KSTGGKAPR)

Implication for layouts:

- Any layout that hard-codes **KSTGGKAPR** (H3K9/K14 region) will not capture the gamete-specific HTR10 N-terminus.

75.2 H3 K27–K36 block: species/isoform signature motifs (verified)

The motif at position ~28 is not universal across AT/MP/CR. The exact “KSAP*” block is a major portability gate for layouts in the 27–40 region.

From `H3_AT_MP_CR.txt`:

Entry	Organism	Motif starting at position 28 (1-based)
sp	P50564	H33_CHLRE
sp	Q42681	H31_CHLRE
sp	Q6LCW8	H32_CHLRE
sp	Q5DWI3	H3_MARPO
AT1G09200.1 (HTR2)	Arabidopsis thaliana	KSAPATGGVKKPHR
AT3G27360.1 (HTR3)	Arabidopsis thaliana	KSAPATGGVKKPHR
AT5G10390.1 (HTR13)	Arabidopsis thaliana	KSAPATGGVKKPHR
AT5G10400.1 (HTR9)	Arabidopsis thaliana	KSAPATGGVKKPHR
AT5G65360.1 (HTR1)	Arabidopsis thaliana	KSAPATGGVKKPHR
AT4G40040.1 (HTR5)	Arabidopsis thaliana	KSAPTTGGVKKPHR

Entry	Organism	Motif starting at position 28 (1-based)
AT5G10980.1 (HTR8)	Arabidopsis thaliana	KSAPTTGGVKKPHR
AT1G19890.1 (HTR10)	Arabidopsis thaliana	— (does not contain this block)

Implication for layouts:

- A layout built on **KSAPATGGVKKPHR** (Arabidopsis H3.1-like) is not sequence-identical to:
 - Arabidopsis H3.3 (**KSAPTTGGVKKPHR**)
 - Marchantia (**KSAPSTGGVKKPHR**)
 - Chlamydomonas (**KTPATGGVKKPHR**)
- Therefore, “portable across species” for the 27–40 region must be treated as false unless you have a separate layout per motif variant.

75.3 H4 motifs: what is conserved, and what changes (verified)

From **H4_AT_MP_CR.txt**:

A) The H4 4–17 N-terminal block is conserved as **GKGGKGLGKGGAKR** in the entries included, starting at position 5.

B) The K20 peptide differs between Arabidopsis and Marchantia in this bundle:

Entry	Organism	K20 peptide (position 21)
AT2G28740.1	Arabidopsis thaliana	KVLR
Mapoly* (multiple)	Marchantia polymorpha	KVFR
sp	P50566	H4_CHLRE

Implication for layouts:

- A strict **KVLR**-based H4K20 layout will not match Marchantia sequences in this bundle (**KVFR**).

C) The H4K44 region **RGGVKR** is conserved in the sequences in this bundle, starting at position 41.

D) The mid-body motif in Arabidopsis/Marchantia is **DAVTYTEHARRK** starting at position 69 (in the sequences where it is present in this bundle).

- Note: one of the H4 audit rows refers to a panel “68–78 (DAVTYTEHAKR)”. Sequence-level evidence in this bundle is **DAVTYTEHARRK**. Treat that panel’s peptide string as a panel label; verify the exact digestion/cleavage form in the MATLAB layout itself before using it as a strict sequence gate.

76. Layout inventory table (grounded in the repo’s audit notes)

This section is a compact index of layout/controller scripts and what they quantify. The “relative_path” values below are copied from the audit TSV-style rows present in:

- **notas_mapping_multi_sp**
- **notas_README_canonical**

If your repository tree differs, treat these paths as identifiers, not guarantees.

76.1 H3 panels (from `notas_mapping_multi_sp`)

Layout file	relative_path (as recorded)	Peptide (as described)	Region	What it quantifies (as described)	Key portanote (sequence level)
H3_01_3_8.m	(described in narrative; path not in TSV row)	TKQTAR	H3 3–8 / N-tail	unmod + K4me1/me2/me3/ac (H3K4 panel)	TKQTAR is conserved across AT/MP/CF bundle; motif starts at position in Arabidopsis H3 entries
H3_02_9_17.m	src/layouts/H3	KSTGGKAPR	H3 9–17	K9/K14 single + combinatorial PTMs (10 states mentioned)	Not valid for Arabidopsis HTR10 (gamete H3) has KSTGGK instead of KSTGGK
H3_02a_9_17.m	src/layouts/H3	KSTGGKAPR	H3 9–17	S10ph-centric panel (S10ph ± K9me/ac ± K14ac)	Same motif constraint as above; position at sequence level across AT/MP/CF where KSTGGK exists
H3_02b_9_17.m	layouts/H3/H3_02b_9_17.m	KSTGGKAPR	H3 9–17	S10ac/S10pr combinatorials with K9/K14	Same motif constraint as above
H3_03_18_26.m	layouts/H3/H3_03_18_26.m	KQLATKAAR	H3 18–26	K18/K23 me1/ac combinatorials + T22 ac/pr cases	Sequence conservation must be checked per target species/is (do not assume)
H3_04a_27_40.m	layouts/H3/H3_04a_27_40.m	KSAPATGGVKKPHR	H3 27–40	S28-centered panel: S28ph/S28ac/S28pr with K27 ladder + selective K36 states	Not portable across AT (KSAPTT) MP (KSAPST)

Layout file	relative_path (as recorded)	Peptide (as described)	Region	What it quantifies (as described)	Key portability note (sequence-level)
					CR (KTPA without separate motif specific labels)
H3_06a_53_63.m	layouts/H3	RYQKSTELLIR	H3 53–63	unmod vs S57pr vs Y54pr (derivatization regioisomers)	Sequence check recommended; do not assume this exact peptide exists unchanged in all species
H3_07_73_83.m	layouts/H3	EIAQDFKTDLR	H3 73–83	K79 unmod + me1/me2/me3/ac	In Arabidopsis H3 entries the bundle motif starts at pos 74
H3_08_117_128.m	layouts/H3	VTIMPKDIQLAR	H3 117–128	unmod vs M120ox vs K122ac	In Arabidopsis H3 entries the bundle motif starts at pos 118

76.2 H4 panels (from [notas_README_canonical](#))

Layout file	relative_path (as recorded)	Peptide (as described)	Region	What it quantifies (as described)	Key portability note (sequence-level)
H4_01_4_17.m	src/panels/H4	GKGGKGLGKGGAKR	H4 4–17	K5/K8/K12/K16 acetylation combinatorics (mono/di/tri/tetra)	Motif is conserved in AT/MP/CR bundle; starts at pos 5
H4_02_20_23.m	src/panels/H4	KVLR	H4 20–23	K20me1/me2/me3/ac	Marchantia sequences in bundle show KVFR instead of KVLR (not sequence-identical)

Layout file	relative_path (as recorded)	Peptide (as described)	Region	What it quantifies (as described)	Key portability note (sequence- level)
H4_02c_20_36.m	matlab/panels/H4	KVLRDNIQGITKPAIRR	H4 20– 36	K20 panel on the Arg-extended peptide	Same KVLRL vs KVFR caveat for Marchantia applies
H4_04_40_45.m	histone/H4	RGGVKR	H4 40– 45	H4K44ac (unmod anchor + K44ac)	Motif is conserved in AT/MP/CR bundle; starts at pos 41
H4_05_68_78.m	histone/H4	DAVTYTEHAKR (panel label)	H4 ~68–78	“Y72pr” / derivatization artifact monitoring (as described)	Sequence bundle shows DAVTYTEHARRK (AT/MP) at pos 69; verify exact peptide form in the MATLAB layout

Operational use:

- This table is meant to be your “panel index” while reading outputs and while writing Methods.
- It is also a pre-flight checklist: if the target species does not contain the exact peptide motif, do not reuse the panel without a species-specific variant.

77. How to use the motif tables to decide “layout portability” (minimal rules)

These rules are conservative and sequence-level only. They do not assume RT transfer.

Rule 1 — Exact peptide string match is mandatory for “same layout, different species”.

- If the peptide differs by even one residue (e.g., KSAPAT... vs KSAPTT...), treat it as a different layout requirement.

Rule 2 — If a species/isoform does not contain the motif, the panel cannot quantify it.

- Example: Arabidopsis HTR10 (MGH3) does not contain the KSAP* 27–40 block, and its K9/K14 peptide is KSTGGKGPR rather than KSTGGKAPR.

Rule 3 — Use `H3_AT_MP_CR.txt` / `H4_AT_MP_CR.txt` as the first gate; do not skip it.

- It prevents “silent wrong quantification” where a panel runs but quantifies noise because the target peptide does not exist.

Rule 4 — Treat “panel labels” (e.g., DAVTYTEHAKR) as labels, not proof.

- Always verify the exact sequence form you expect at the digestion level by reading the MATLAB layout and confirming the peptide string it encodes in its His/pep_seq definitions.

78. Suggested next section to write (consistent with this manual's flow)

Next section title (recommended): “Panel cards: one-page spec per layout (peptide, marks, anchor, relocation, outputs, portability)”

Why this is the logical next step:

- You now have (i) verified motif gates and (ii) a grounded inventory of panels.
- What is still missing for real users is a consistent “panel card” format they can read quickly.

Minimum fields for each panel card:

1. Layout file name (+ where it writes its `.mat`)
2. Peptide sequence and region label (H3/H4 start–end)
3. PTM rows (exact states, as encoded in `mod_type/mod_short`)
4. Anchor definition (what is “unmod” in that panel; what it assumes about derivatization)
5. Relocation strategy (MS1-only vs MS1+MS2, which helpers are used)
6. Output artifacts (figures, `.mat`, optional PSM exports)
7. Portability gate (exact motif + known exceptions from Section 75)

If you want the manual to stay “no invention”, the clean way is:

- build each card by extracting directly from the corresponding `.m` file header/comments (and from the audit notes when available).

79. Panel cards (layouts): a “one page per peptide” index you can trust

This section turns the existing audit notes into panel “cards” with consistent fields:

- peptide string (the primary key for the layout)
- histone coordinates (as stated in the notes)
- PTM rows (what the panel quantifies)
- RT anchoring strategy (what defines the RT zero-point for the panel)
- relocation + extraction helpers (which low-level functions the panel delegates to)
- outputs (what files you should expect after a run)
- portability gate (when the same panel is safe to reuse across species/isoforms)

Source of truth for this section:

- `notas_mapping_multi_sp` (H3 panels)
- `notas_README_canonical` (H4 panels)

79.1 Panel card template (canonical fields)

For each panel, document the same minimal spec:

A) Panel ID

- MATLAB file: `H3_XX_...m` / `H4_XX_...m`
- I/O signature (as recorded in notes)

B) Peptide + coordinates

- `His.pep_seq`
- region label (e.g., “H3 9–17”)

C) PTM rows

- list of PTM states (what appears as `His.mod_short` / `His.mod_type`)

D) RT anchor and relocation

- what is the anchor species (“unmod”, “S10ph”, etc.)
- whether it uses MS1-only relocate or MS2-assisted relocate (DA)
- whether it uses `find_pair` (explicit doublet disambiguation)

E) Quantification helpers (named)

- which `get_histone*` functions are used for extraction

F) Expected outputs

- `<panel>.mat`
- layout figures (XIC/layout plots)
- optional `.plabel` (PSM exports) when `special.nDAmode==1` (only when notes explicitly say so)

G) Portability gate

- “same peptide string” requirement, plus known isoform exceptions from Section 75

79.2 H3 panel inventory (verified from `notas_mapping_multi_sp`)

The goal of this table is fast navigation. It is not a substitute for reading the layout `.m`, but it prevents common mistakes (wrong peptide, wrong isoform, wrong anchor).

Panel	Peptide (<code>His.pep_seq</code>)	Region	Anchor concept	PTMs (rows)
H3_01_3_8	TKQTAR	H3 1–6 (N-term; “K4 panel”)	unmod	unmod; K4me1; K4me2; K4me3; K4ac
H3_02_9_17	KSTGGKAPR	H3 9–17 (K9/K14)	unmod	unmod; K9me1/2/3/ac; K14ac; K9me(1/2/3)+K14ac; K9ac+K14ac (10 states total)
H3_02a_9_17	KSTGGKAPR	H3 9–17 (S10ph-centric)	S10ph	S10ph is mandatory in every row; combined with K9 (me1/2/3/ac) and/or K14ac
H3_02b_9_17	KSTGGKAPR	H3 9–17 (S10ac/S10pr- centric)	S10ac	S10ac / S10pr plus combinations with K9(me1/2/3/ac) and K14ac (11 rows listed in notes)

Panel	Peptide (His.pep_seq)	Region	Anchor concept	PTMs (rows)
H3_03_18_26	KQLATKAAR	H3 18–26 (K18/T22/K23)	unmod	K18me1/ac; K23me1/ac; K18me1K23me1; K18acK23ac; T22ac/pr; K18acT22pr (T22* often “display off” in plots per notes)
H3_04a_27_40	KSAPATGGVKKPHR	H3 27–40 (S28- centered)	inherited RTs	S28ph/S28ac/S28pr combined with K27 ladder (+ selected K36 states); quantification delegated to get_histone12
H3_06a_53_63	RYQKSTELLIR	H3 53–63 (derivatization regioisomers)	unmod	unmod; S57pr; Y54pr
H3_07_73_83	EIAQDFKTDLR	H3 73–83 (K79 panel)	unmod	unmod; K79me1; K79me2; K79me3; K79ac
H3_08_117_128	VTIMPKDIQLAR	H3 117–128 (C- term panel)	unmod	unmod; M120ox; K122ac

Portability reminder (sequence-level):

- [KSTGGKAPR](#) is not valid for Arabidopsis gamete H3 (HTR10 / ATMGH3): it carries [KSTGGKGPR](#) in the notes.
- [KSAPATGGVKKPHR](#) is H3.1-like; H3.3 in Arabidopsis uses [KSAPTTGGVKKPHR](#); Marchantia uses [KSAPSTGGVKKPHR](#); Chlamydomonas uses [KTPATGGVKKPHR](#).

79.3 H4 panel inventory (verified from [notas_README_canonical](#))

Panel	Peptide (His.pep_seq)	Region	Anchor concept	PTMs (rows)
H4_01_4_17	GKGGKGLGKGGAKR	H4 4–17	unmod	unmod (derivatized “pr”) + full K5/K8/K12/K16 acetylation combinatorics (mono/di/tri/tetra)
H4_02_20_23	KVLR	H4 20–23	unmod	unmod + K20me1/me2/me3/ac; explicit me2 vs me3 disambiguation with find_pair noted
H4_02c_20_36	KVLRDNIQGITKPAIRR	H4 20–36 (Arg- extended)	unmod	unmod + K20me1/me2/me3/ac (auxiliary/confirmatory; display=0 noted)
H4_04_40_45	RGGVKR	H4 40–45	unmod	unmod + K44ac; K44ac expected earlier than unmod (notes emphasize checking RT cleanliness)

Panel	Peptide (His.pep_seq)	Region	Anchor concept	PTMs (rows)
H4_05_68_78	DAVTYTEHAKR (panel label)	H4 68–78	unmod	unmod + Y72pr (O-propionyl artifact monitoring; Y72pr expected later than unmod)

Portability reminder:

- Marchantia in your H4 bundle uses **KVFR** where Arabidopsis/Chlamydomonas use **KVLR**. Treat **H4_02_20_23** as non-portable to Marchantia unless you have a motif-specific variant.

79.4 Panel cards (H3) — verified summaries, no extra assumptions

79.4.1 H3_01_3_8 (H3K4 panel)

Panel ID

- **H3_01_3_8.m**
- Signature:
H3_01_3_8(MS1_index,MS1_peaks,MS2_index,MS2_peaks,ptol,cur_outpath,special)

Peptide + region

- **TKQTAR** (6 aa)
- described as “H3 1–6, target K4” in notes

PTM rows

- unmod
- K4me1 / K4me2 / K4me3 / K4ac

RT anchor + relocation

- anchor: unmod
- notes explicitly state: unmod is extracted with **get_histone0**, then modified forms are re-centered relative to the unmod anchor and relocated.

Extraction helpers (as named)

- **get_histone0** for unmod
- **get_histone1** for K4me1/2/3/ac

Outputs

- **H3_01_3_8.mat**
- diagnostic plots via **draw_layout**
- optional **.plabel** via **GetPSM** when **special.nDAmode==1** (notes explicitly mention this behavior)

Portability gate

- peptide-level; safe only when **TKQTAR** exists exactly in the target isoform.

79.4.2 H3_02_9_17 (K9/K14 panel; 10-state core)

Panel ID

- `H3_02_9_17.m`
- Signature recorded in notes; output: `H3_02_9_17.mat` + `.plabel` + plots

Peptide + region

- `KSTGGKAPR` (H3 9–17)
- residue mapping per notes:
 - peptide position 1 corresponds to H3K9
 - peptide position 6 corresponds to H3K14

PTM rows (10-state panel listed in notes)

1. unmod
2. K9me1
3. K9me2
4. K9me3
5. K9ac
6. K14ac
7. K9me1K14ac
8. K9me2K14ac
9. K9me3K14ac
10. K9acK14ac

RT anchor + relocation

- anchor: unmod
- then relocation of modified forms around that anchor (MS1 or MS2 assisted; notes frame this as “standard EpiProfile relocation logic”)

Extraction helpers (as named)

- panel orchestrates anchor + relocation; extraction is via the usual `get_histone*` family (details are in the layout itself; the notes emphasize preservation of the original logic).

Outputs

- `H3_02_9_17.mat`, plots, optional `.plabel`

Portability gate

- not valid for Arabidopsis HTR10/ATMGH3 in your bundle (variant peptide `KSTGGKGPR`).

79.4.3 H3_02a_9_17 (S10ph-centric twin of the K9/K14 panel)

Panel ID

- `H3_02a_9_17.m`

Peptide + region

- **KSTGGKAPR** (H3 9–17)

PTM rows (conceptual)

- S10ph is mandatory in every row (peptide position 2)
- combined with K9: me1/me2/me3/ac
- combined with K14: ±K14ac

RT anchor + relocation

- anchor: S10ph-only (this is the “unmod equivalent” for this panel)
- notes explicitly describe it as “phospho-centric twin”, same relocation logic family as H3_02_9_17

Helpers (as named in notes)

- relocation helpers: `get_rts` / `get_rts2` / `find_pair_new`
- extraction helpers named: `get_histone0/1/2/11` (as recorded)

Outputs

- `.mat` + plots (and `.plabel` if nDAmode logic is enabled in the file, per the family pattern)

Portability gate

- peptide-level: requires exact **KSTGGKAPR**.

79.4.4 H3_02b_9_17 (S10ac / S10pr panel)

Panel ID

- **H3_02b_9_17.m**

Peptide + region

- **KSTGGKAPR** (H3 9–17)
- residue mapping per notes:
 - peptide pos 1 = H3K9
 - peptide pos 2 = H3S10
 - peptide pos 6 = H3K14

PTM rows (11 entries listed explicitly in notes)

- S10ac
- S10pr
- K9me1S10ac
- K9me2S10ac
- K9me3S10ac
- K9acS10ac
- S10acK14ac
- S10prK14ac
- K9me2S10acK14ac
- K9me3S10acK14ac
- K9acS10acK14ac

RT anchor + relocation

- anchor: S10ac (“anchor #1” in notes)
- uses relocate/relocate2 family and specialized extractors for this peptide

Helpers (as named)

- extraction via `get_histone0/1/2/10/11` (as recorded)

Outputs

- `.mat`, plots, optional `.plabel` (family pattern; verify the file for the exact flag)

Portability gate

- peptide-level: requires exact `KSTGGKAPR`.

79.4.5 H3_03_18_26 (K18/T22/K23 panel)

Panel ID

- `H3_03_18_26.m`

Peptide + region

- `KQLATKAAR` (H3 18–26)

PTM rows (as summarized in its audit row)

- K18me1, K18ac
- K23me1, K23ac
- K18me1K23me1
- K18acK23ac
- T22ac / T22pr and K18acT22pr (quantified; T22* often treated as “display off” per notes)

RT anchor + relocation

- anchor: unmod
- notes emphasize pair-resolution cases (K23me1 vs K18me1; T22ac vs K18acT22pr) and “paired extraction” for K23ac/K18ac.

Helpers (as named in the notes)

- extraction switches between `get_histone10` vs `get_histone2` depending on RT separation; plus standard `get_histone` calls for other rows

Outputs

- `H3_03_18_26.mat`, plots, optional `.plabel` when `nDAmode==1` (audit row explicitly mentions it)

Portability gate

- only when `KQLATKAAR` exists exactly in the target isoform/species.

79.4.6 H3_04a_27_40 (S28-centered child panel using inherited RTs)

Panel ID

- `H3_04a_27_40.m`

Peptide + region

- `KSAPATGGVKKPHR` (H3 27–40)

PTM rows (as summarized in the audit row)

- S28ph, S28ac, S28pr
- combined with K27me1/2/3 ladder
- selected K36 states (explicitly noted: K36me2 and a specific K27me3S28acK36me1 case)
- S28pr is quantified but `display=0` (hidden in plots) per notes.

RT anchor + relocation

- key point from notes: it does NOT re-anchor on unmod.
- it imports RT seeds from `H3_04_27_40.xls` (`[rt]`) and uses those as `ref_rts`
- uses `find_pair` for specific doublets (K27me2 vs K27me3) in phospho/acetyl contexts

Helpers (as named)

- extraction delegated to `get_histone12` for rows 1–11 (as written in notes)

Outputs

- `H3_04a_27_40.mat`, plots, optional `.plabel`

Portability gate

- not portable across motif variants (`KSAPAT...` vs `KSAPTT...` vs `KSAPST...` vs `KTPAT...`). Use a motif-specific layout per sequence.
-

79.4.7 H3_06a_53_63 (derivatization regioisomer monitor)

Panel ID

- `H3_06a_53_63.m`

Peptide + region

- `RYQKSTELLIR` (H3 53–63)

PTM rows

- unmod (derivatized baseline)
- S57pr
- Y54pr

RT anchor + relocation

- anchor: unmod
- notes describe explicit disambiguation of S57pr vs Y54pr via windows/intensity logic or a joint extractor.

Helpers (as named)

- mentions `get_histone2` as a joint extractor route; also uses standard relocate + extraction family

Outputs

- `H3_06a_53_63.mat`, plots

Portability gate

- peptide-level; verify motif presence in the target isoform/species.
-

79.4.8 H3_07_73_83 (H3K79 core panel)

Panel ID

- `H3_07_73_83.m`

Peptide + region

- `EIAQDFKTDLR` (H3 73–83)

PTM rows (5-state panel explicitly listed)

- unmod
- K79me1
- K79me2
- K79me3
- K79ac

RT anchor + relocation

- anchor: unmod
- drift correction + relocation around the anchor (notes explicitly describe this pattern)

Helpers (as named)

- extraction for modified rows via `get_histone1` (explicit in the audit row)

Outputs

- `H3_07_73_83.mat`, plots

Portability gate

- peptide-level; requires exact `EIAQDFKTDLR`.
-

79.4.9 H3_08_117_128 (H3 C-terminal panel: M120ox + K122ac)

Panel ID

- `H3_08_117_128.m`

Peptide + region

- `VTIMPKDIQLAR` (H3 117–128)

PTM rows

- unmod
- M120ox
- K122ac

RT anchor + relocation

- anchor: unmod
- notes provide explicit window logic:
 - M120ox window is earlier than unmod (unmod-10 .. unmod-0.1)
 - K122ac window is constrained between M120ox+0.1 and unmod-0.1 (as recorded)

Helpers (as named)

- extraction via `get_histone1` for modified rows (explicit)

Outputs

- `H3_08_117_128.mat`, plots

Portability gate

- peptide-level; verify exact motif.
-

79.5 Panel cards (H4) — verified summaries, no extra assumptions

79.5.1 H4_01_4_17 (H4 N-terminus Kac combinatorics)

Panel ID

- `H4_01_4_17.m`

Peptide + region

- `GKGGKGLGKGGAKR` (H4 4–17)

PTM rows

- unmod (derivatized baseline)
- all mono/di/tri/tetra acetylation states over K5/K8/K12/K16

RT anchor + relocation

- anchor: unmod
- relocation via `relocate/relocate2` (DA noted)
- notes also mention `change_order` for tri-acetyl row ordering (plot clarity)

Helpers (as named)

- extraction via `get_histone0/1/4/6`

Outputs

- `H4_01_4_17.mat`, plots, optional `.plabel` per the common panel pattern

Portability gate

- peptide-level; safe only when peptide exists exactly.
-

79.5.2 H4_02_20_23 (H4K20 short peptide panel)

Panel ID

- `H4_02_20_23.m`

Peptide + region

- `KVLR` (H4 20–23)

PTM rows

- unmod
- K20me1, K20me2, K20me3, K20ac

RT anchor + relocation

- anchor: unmod via `check_ref + get_histone0`
- explicit doublet handling: `find_pair` for me2 vs me3
- notes describe a special fallback when `ndebug==2` (joint search unmod + me1)

Helpers (as named)

- extraction of each PTM row with `get_histone1` (explicit in the audit row)

Outputs

- `H4_02_20_23.mat`, plots, optional `.plabel`

Portability gate

- Marchantia in your H4 bundle uses `KVFR`, so this exact layout is not sequence-identical there.
-

79.5.3 H4_02c_20_36 (H4K20 Arg-extended auxiliary panel)

Panel ID

- `H4_02c_20_36.m`

Peptide + region

- `KVLRDNIQGITKPAIRR` (H4 20–36)

PTM rows

- unmod + K20me1/me2/me3/ac
- marked as auxiliary/confirmatory; `display=0` in the audit row

RT anchor + relocation

- anchor: unmod; drift correction; relocation in the standard pattern

Helpers (as named)

- uses `get_histone0/get_histone1` plus the standard relocation helpers

Outputs

- `H4_02c_20_36.mat`, plots, optional `.plabel`

Portability gate

- peptide-level: requires exact peptide match.
-

79.5.4 H4_04_40_45 (H4K44ac panel)

Panel ID

- `H4_04_40_45.m`

Peptide + region

- `RGGVKR` (H4 40–45)

PTM rows

- unmod
- K44ac

RT anchor + relocation

- anchor: unmod (`check_ref` + `get_histone0`)
- notes specify K44ac is relocated earlier than unmod (window `[RT(unmod)-14, RT(unmod)-0.1]`)

Helpers (as named)

- extraction via `get_histone1` for K44ac

Outputs

- `H4_04_40_45.mat`, plots, optional `.plabel`

Portability gate

- peptide-level; requires exact match.
-

79.5.5 H4_05_68_78 (Y72 O-propionylation artifact monitor)

Panel ID

- `H4_05_68_78.m`

Peptide + region

- panel label: `DAVTYTEHAKR` (H4 68–78)

PTM rows

- unmod
- Y72pr (O-propionylation artifact; expected later RT than unmod)

RT anchor + relocation

- anchor: unmod
- notes specify Y72pr is relocated later than unmod: window `[RT(unmod)+0.1, RT(unmod)+30]`

Helpers (as named)

- extraction via `get_histone1` for Y72pr

Outputs

- `H4_05_68_78.mat`, plots, optional `.plabel`

Portability gate

- peptide-level; confirm exact motif in the target species/isoform.

80. Suggested next section (what to write next, and why)

Recommended title: “From panel `.mat` to cohort export: the output contract and validation checks”

Reason:

- You now have a verified panel index (what each layout is supposed to produce).
- The next failure mode in real use is not “wrong panel” but “wrong parsing”: mixing export formats, stale caches, or broken invariants.

What that next section should contain (grounded target fields; no assumptions beyond what the notes repeatedly mention):

1. Minimal `.mat` contents to expect per panel
 - `His` struct (panel definition)
 - `pep_rts` and `pep_intens` (RTs and intensities per PTM × charge)
 - `mono_isointens` (used by `draw_layout` in multiple notes)
2. How panels roll up into a cohort table
 - which script/function writes the combined export (to be pinned from MATLAB sources)
3. Export format detection rules (Pattern A vs Pattern B, as already documented)
4. Validator checks you can run before downstream stats
 - sample count consistency
 - peptide row presence
 - RT sanity ranges
 - empty-window / zero-intensity detection

81. What a “layout panel” produces (verified output contract)

Across panels, the skeleton is consistent (the details differ per peptide/PTM set):

1. `His = init_histone();`

Builds a local catalog for one peptide region (sequence, PTM rows, charges, m/z, RT seeds, display flags).

2. `[pep_rts, pep_intens, mono_isointens] = calculate_layout(..., His, special);`

Quantifies each PTM row (and each charge) using RT-guided XIC extraction.

3. `output_histone(...);`

Writes a panel `.mat` that (at minimum) contains:

- `His`
- `pep_intens`
- `pep_rts`

4. `draw_layout(...);`

Uses:

- `mono_isointens` (monoisotopic intensity profile across MS1 scans)
- `isorts` (RT per MS1 scan) to draw the panel plots (XICs + RT markers).

5. Optional (only when enabled):

If `special.nDAmode == 1`, the panel can call `GetPSM(...)` and write a simplified `.plabel` file for external viewers.

This is explicitly described in the H3 multi-species notes (`notas_mapping_multi_sp`) and repeated in the H4 canonical notes (`notas_README_canonical`).

82. What to inspect first inside a panel `.mat` (verified fields)

The first thing to check is the `His` struct. It is the “truth table” for that panel.

Common `His` fields (observed in the notes):

- `His.pep_seq`
The peptide sequence used as the panel key (exact string match matters).
- `His.mod_short`
Human labels for each PTM row, e.g. `unmod`, `K4me1`, `K4ac`.
- `His.mod_type`
The machine encoding for the same PTM rows (see Section 84).
- `His.pep_ch`
Charge states used for extraction, stored as a matrix `[#PTM rows × #charges]`.
- `His.pep_mz`
Theoretical m/z values derived from `His.pep_seq + His.mod_type + His.pep_ch`.
- `His.rt_ref`
RT “seed” values (minutes) per PTM row. These are adjusted by drift correction after anchoring the `unmod` row.
- `His.display`
Plot-control flags; some rows are quantified but hidden in plots (`display=0` is used in at least one

H4K20 auxiliary panel and some specialized rows in H3/H4 notes).

Then inspect the numeric outputs:

- `pep_rts`
RT per PTM row × charge.
- `pep_intens`
Integrated intensity (AUC) per PTM row × charge.

Important: `mono_isointens` is a compute-time matrix used for plotting. The notes state that `output_histone` packs `His`, `pep_intens`, and `pep_rts` into the `.mat`. Do not assume `mono_isointens` is saved unless you confirm in your specific `output_histone.m` implementation.

83. How RT anchoring + drift correction works (worked example: H3_01_3_8, verified)

The H3K4 panel (H3_01_3_8, peptide `TKQTAR`) is documented in detail in `notas_mapping_multi_sp`. It shows the exact strategy used by many panels (with peptide-specific window choices).

83.1 Shapes of the core matrices (verified)

Inside `calculate_layout`:

- `npep, ncharge = size(His.pep_mz)`
- `num_MS1 = size(MS1_index, 1)`

Then:

- `pep_rts = zeros([npep, ncharge])`
- `pep_intens = zeros([npep, ncharge])`
- `mono_isointens = zeros([num_MS1, npep])`

For H3_01_3_8 specifically:

- `npep = 5` (unmod + 4 PTMs)
- `ncharge = 2`
- `mono_isointens` has dimension `[MS1 scans × PTM rows]`

83.2 Anchor on `unmod` (verified)

The panel stores:

- `His.rt_unmod_orig = His.rt_ref(1)`

Then (unless in “pure debug” mode) it runs `check_ref(...)` to refine the expected RT seed for the unmodified peptide.

If RT guidance is unstable (notes describe a “debug 2” state), the code broadens the search using `get_rts(...)` or `get_rts2(...)` (MS2-assisted mode) across a wide RT range, using a related PTM row to guide where to anchor.

83.3 Extract unmod with `get_histone0`, then apply drift correction (verified)

The panel calls:

- `[cur_rts, cur_intens, cur_mono_isointens] = get_histone0(...)`

If `cur_rts(1) > 0` (a real RT was found):

- `His.rt_ref(1) = cur_rts(1)`
- `delta = cur_rts(1) - His.rt_unmod_orig`
- `His.rt_ref(2:end) = His.rt_ref(2:end) + delta`

Then it stores unmod results into:

- `pep_rts(1, :)`
- `pep_intens(1, :)`
- `mono_isointens(:, 1)`

Interpretation:

- the panel assumes the relative elution ordering between PTM rows is stable enough that one global drift `delta` can shift all seed RTs.

83.4 Relocate each PTM row, then extract with `get_histone1` (verified)

After anchoring, the panel updates RT seeds with one of:

- `relocate(...)` (MS1-only using `get_rts`)
- `relocate2(...)` (MS2-assisted using `get_rts2` / `get_rts22`)
- `relocated(...)` (a special debug relocation mode referenced in the notes)

Then, for each PTM row (e.g., rows 2..5 in H3_01_3_8):

- `[cur_rts, cur_intens, cur_mono_isointens] = get_histone1(...)`
- If `cur_rts(1) > 0`, store into `pep_rts, pep_intens, mono_isointens`.

83.5 Why `find_pair` exists (verified purpose)

Some PTMs are treated as a coupled pair (example in H3_01_3_8: K4me2 and K4me3). The notes show:

- both are searched in the same RT window
- `find_pair(...)` resolves two coherent RTs from overlapping candidate lists using intensity and top-peak logic

That pattern reappears in other panels when isobaric or near-isobaric states compete.

83.6 The “timeline” idea (verified for H3_01_3_8)

H3_01_3_8 uses explicit RT windows relative to the anchored unmod RT (the exact bounds are panel-specific). The notes document a conceptual ordering:

- some methylated states are expected before unmod
- some acetylated states are expected between a methylated state and unmod
- some states are expected after unmod

Do not reuse H3_01_3_8's numeric window bounds for other peptides. Reuse the concept: “anchor → drift → relocate windows → extract”.

84. `mod_type` dialect (verified minimal grammar)

`His.mod_type` encodes modifications as a semicolon-separated list of tokens.

Each token has:

- a position index, and
- a modification code.

Examples explicitly documented in the notes:

A) H3_01_3_8 (`His.pep_seq = 'TKQTAR'`)

- `0,pr;2,pr;` (unmod row: N-term propionylated; Lys position 2 propionylated)
- `0,pr;2,me1;` (K4me1 row)
- `0,pr;2,ac;` (K4ac row)

Here, “2” means “second residue in the peptide string”. In `TKQTAR`, residue 2 is the K (the H3K4 site).

B) H4_05_68_78 (`His.pep_seq = 'DAVTYTEHAKR'` as panel label)

- `0,pr;10,pr;` for unmod (N-term + Lys(10) propionylated)
- `0,pr;10,ypr;` for the Y72pr artifact row (the notes describe this as O-propionylation tracking)

C) H4_02c_20_36 (`His.pep_seq = 'KVLRDNIQGITKPAIRR'`) The notes describe the pattern:

- unmod uses propionylation at the N-terminus and at the Lys positions in the peptide
- K20 PTM rows replace the Lys(1) token with `me1/me2/me3/ac`

Practical meaning:

- `mod_type` is the single source of truth for:
 - theoretical mass (via `calculate_pepmz(His) / GetMods`)
 - which residue index is being modified
 - whether the “unmod” row is interpreted as “fully derivatized baseline”

Rule: if you change `His.pep_seq`, you must re-check every residue index used in `His.mod_type`. Off-by-one errors here silently destroy quantification.

85. Peptide inventory tables for portability (verified from repo bundles)

The repo ships FASTA-like bundles that you can treat as the first-pass gate before you reuse a panel across species/isoforms:

- `H3_AT_MP_CR.txt`
- `H4_AT_MP_CR.txt`

- [notas_README_canonical](#) includes a worked H4 table (copied below)

85.1 H4 peptide inventory (as documented in [notas_README_canonical](#))

peptide	H4_CHLRE	H4_AT	H4_MARPO	comment
GKGGKGLGKGGAKR	yes (5–18)	yes (5–18)	yes (5–18)	H4 N-term core peptide
HRKVLRL	yes (19–24)	yes (19–24)	no (HRKVFR)	Marchantia L→F
KVLRL	yes (21–24)	yes (21–24)	no (KVFR)	Marchantia L→F
HRKVFR	no	no	yes	Marchantia-specific
KVFR	no	no	yes	Marchantia-specific
DNIQGITKPAIR	yes (25–36)	yes (25–36)	yes (25–36)	shared central peptide
KVLRDNIQGITKPAIR	yes (21–35)	yes (21–35)	no	AT+Chlamy, not MARPO
KVLRDNIQGITKPAIRR	yes (21–36)	yes (21–36)	no	AT+Chlamy, not MARPO
KVFRDNIQGITKPAIR(R)	no	no	yes	Marchantia version
RGGVKR	yes (41–46)	yes (41–46)	yes (41–46)	conserved motif
DSVTYTEHARR	yes	no	no	Chlamy-specific
DAVTYTEHARR	no	yes	yes	AT+MARPO
KTVTAMDVVYALKR	yes (80–93)	yes (80–93)	yes (80–93)	shared C-term peptide

Immediate consequences for panels:

- Any H4K20 panel built on [KVLRL](#) is not sequence-identical in Marchantia ([KVFR](#)). Treat it as non-portable unless you build a Marchantia-specific variant.

85.2 H3 peptide inventory (extracted from [H3_AT_MP_CR.txt](#) bundle)

Representative records used here:

- CHLRE_H3: Chlamydomonas H3 (example entry in the bundle)
- MARPO_H3: Marchantia H3
- AT_H3.1: Arabidopsis H3.1 (HTR2 example)
- AT_H3.3: Arabidopsis H3.3 (HTR5 example)
- AT_MGH3: Arabidopsis male-gamete H3 (HTR10 / ATMGH3)

Positions below are 1-based positions in the full protein sequence from the bundle.

peptide / family	CHLRE_H3	AT_H3.1	AT_H3.3	AT_MGH3	MARPO_H3	comment
TKQTAR	yes (4)	yes (4)	yes (4)	yes (4)	yes (4)	core N-term motif used by H3K4 panel
KSTGGKAPR	yes (10)	yes (10)	yes (10)	no	yes (10)	core K9/K14 peptide; absent in AT_MGH3

peptide / family	CHLRE_H3	AT_H3.1	AT_H3.3	AT_MGH3	MARPO_H3	comment
KSTGGKGPR	no	no	no	yes (10)	no	AT_MGH3-specific variant of K9/K14 region
KQLATKAAR	yes (19)	yes (19)	yes (19)	no	no	H3 18–26 panel peptide (AT/Chlamy)
KQLASKAAR	no	no	no	no	yes (19)	Marchantia variant (H3 18–26 family)
KSAPATGGVKKPHR	no	yes (28)	no	no	no	Arabidopsis H3.1-like 27–40
KSAPTTGGVKKPHR	no	no	yes (28)	no	no	Arabidopsis H3.3-like 27–40
KSAPSTGGVKKPHR	no	no	no	no	yes (28)	Marchantia 27–40
KTPATGGVKKPHR	yes (28)	no	no	no	no	Chlamydomonas 27–40
KTRRPYRGGVKRAHR	no	no	no	yes (28)	no	AT_MGH3-specific 27–40 family (gamete H3)
EIAQDFKTDLR	yes (73)	yes (74)	yes (74)	no	no	H3K79 panel peptide (AT/Chlamy)
EIAQDFKSDLR	no	no	no	no	yes (74)	Marchantia variant (S instead of T)
EIAQDFKVDLR	no	no	no	yes (75)	no	AT_MGH3 variant (V instead of T)
VTIMPKDIQLAR	yes (117)	yes (118)	yes (118)	no	no	H3 C-term panel peptide (AT/Chlamy)
VTIQSKDIQLAR	no	no	no	no	yes (118)	Marchantia variant
VTIMSKDIQLAR	no	no	no	yes (119)	no	AT_MGH3 variant

Immediate consequences for panels:

- Many “canonical H3” panels are not portable to AT_MGH3 without separate motif-specific layouts (K9/K14, 18–26, 27–40, K79 region, C-terminus all diverge in this bundle).
- Marchantia requires motif-specific variants for at least the 18–26, 27–40, K79 region, and C-terminus families.

86. “Core” vs “signature” peptides (verified concept from [notas_README_canonical](#))

The notes define a practical split that you can directly use to organize multi-species work:

A) Core multi-species peptides

These are good candidates for shared QC normalization and for “portable panels” (if the peptide string matches).

- H4 (explicitly listed as core in notes):
[GKGGKGLGKGGAKR](#), [DNIQGITKPAIR](#), [RGGVKR](#), [KTVTAMDVVYALKR](#)
- H3 (notes list examples; bundle extraction confirms several):
[TKQTAR](#) is shared; others are shared across subsets (AT + Chlamy; AT + MP; etc.).

B) Signature peptides

These encode isoform/species identity (and force panel duplication).

The notes call out the H3 27–40 family as the clearest case:

- AT_H3.1: [KSAPATGGVKKPHR](#)
- AT_H3.3: [KSAPTTGGVKKPHR](#)
- MARPO: [KSAPSTGGVKKPHR](#)
- CHLRE: [KTPATGGVKKPHR](#)
- AT_MGH3: [KTRRPYRGGVKRAHR](#) (different family)

Operational rule:

- If a peptide is “signature”, do not attempt to reuse the same [.m](#) panel across species/isoforms. Clone and specialize the panel at the peptide-definition level ([His.pep_seq](#), [mod_type](#) indices, theoretical m/z, RT seeds).

87. Minimal “panel portability” checklist (sequence + layout-level)

Do these in order. Stop at the first “no”.

1. Sequence gate (FASTAs):

- Does [His.pep_seq](#) appear exactly in the target protein sequence?
- If not, this panel is non-portable.

2. Residue-index gate ([mod_type](#)):

- Are the residue positions in [His.mod_type](#) still pointing to the correct site within the peptide?
- If the peptide changes by 1 residue, your [mod_type](#) indices can become wrong even if the peptide still contains the target lysine.

3. Mass gate (m/z):

- Recompute theoretical m/z and confirm it matches the expected peptide family.
- This matters most when switching between motif variants (e.g., AT H3.1 vs H3.3 in 27–40).

4. Only after (1–3): RT gate:

- RT seeds (`His.rt_ref`) are dataset- and LC-method-dependent. Expect to retune them even when the peptide is conserved.

88. Suggested next section (still grounded): “From layouts to a validated cohort export”

This manual is now ready for the next hard step, but it requires one missing fact from your repo checkout:

- Which MATLAB function/script aggregates panel `.mat` files into the combined cohort export (the `histone_ratios*.xls` / TSV-like output).

Once that writer is located, the next section should include:

A) Exact output paths and filenames (writer-dependent)

- Where the cohort export is written (relative to `raw_path`)
- What it is called (exact name patterns)

B) Output schema (no ambiguity)

- Required columns/blocks (Ratio / Area / RT)
- Header style (classic “row blocks” vs “2-row header”)
- Invariants (sample count, peptide keys, metric labels)

C) A validator checklist you can run before downstream stats

- “all samples present”
- “no mixed header formats”
- “expected peptide keys exist”
- “RT ranges plausible”
- “no stale layout caches reused accidentally”

If you want this to stay strictly “no invention”, the next action is mechanical:

- search your MATLAB tree for `output_histone` calls and for the string `histone_ratios`
- identify the single writer that creates the combined export
- then document it line-by-line as the cohort export contract.

88. Suggested next section (still grounded): “From layouts to a validated cohort export”

This manual is now ready for the next hard step, but it requires one missing fact from your repo checkout:

- Which MATLAB function/script aggregates panel `.mat` files into the combined cohort export (the `histone_ratios*.xls` / TSV-like output).

Once that writer is located, the next section should include:

A) Exact output paths and filenames (writer-dependent)

- Where the cohort export is written (relative to `raw_path`)
- What it is called (exact name patterns)

B) Output schema (no ambiguity)

- Required columns/blocks (Ratio / Area / RT)
- Header style (classic “row blocks” vs “2-row header”)
- Invariants (sample count, peptide keys, metric labels)

C) A validator checklist you can run before downstream stats

- all samples present
- no mixed header formats
- expected peptide keys exist
- RT ranges plausible
- no stale layout caches reused accidentally

If you want this to stay strictly “no invention”, the next action is mechanical:

- search your MATLAB tree for `output_histone` calls and for the string `histone_ratios`
- identify the single writer that creates the combined export
- then document it line-by-line as the cohort export contract.

Locate the cohort export writer (mechanical)

Goal: find the *single* MATLAB function that writes the combined cohort export (the file usually called something like `histone_ratios.xls`, often TSV-in-disguise).

This section is deliberately mechanical: do the searches, identify the writer, then document it as the export contract.

Fast search (recommended: ripgrep)

Run these from the repository root:

```
# 1) Find any reference to the export name
rg -n "histone_ratios" -S

# 2) Find calls to the exporter wrapper (common name in EpiProfile codebases)
rg -n "\boutput_histone\b" -S

# 3) Find where any .xls is written
rg -n "\.xls" -S

# 4) Find file-writing patterns (TSV writers typically use fopen/fprintf)
rg -n "\bfopen\b|\bfprintf\b|\bfclose\b" -S

# 5) If you suspect MATLAB Excel writing
rg -n "\bxlswrite\b|\bwritetable\b" -S
```

If `rg` is not available, use `grep`:

```
grep -RIn --line-number "histone_ratios" .
grep -RIn --line-number "\boutput_histone\b" .
grep -RIn --line-number "\.xls" .
grep -RIn --line-number "\bfopen\b|\bfprintf\b|\bfclose\b" .
grep -RIn --line-number "\bxlswrite\b|\bwritetable\b" .
```

If the repo has multiple MATLAB roots

Do not assume a fixed folder name. First find where `.m` files live:

```
find . -type f -name "*.m" | head
```

Then constrain searches to that subtree if you want:

```
# example only; replace ./MATLAB_ROOT with your real path
rg -n "histone_ratios" ./MATLAB_ROOT -S
```

How to decide “this is the writer”

You want a file that satisfies **both**:

1. It contains the string `histone_ratios` (or constructs that filename), and
2. It performs the actual write (any of: `fopen/fprintf`, `xlswrite`, `writetable`, `dlmwrite`, etc.)

Typical patterns you’ll see in the true writer:

- a filename like `histone_ratios.xls` (or `histone_ratios_*.xls`)
- a delimiter such as `\t` or `\n`
- two header rows being printed (sample names row + metric labels row)
- a loop over “features/peptides/peptideforms”
- repeated blocks per sample: `Ratio`, `Area`, `RT(min)` (or a subset)

Confirm it’s *the* combined export (not per-run detail)

Per-run artifacts live under:

- `<raw_path>/histone_layouts/<prefix>_<raw_name>/`
- `<raw_path>/histone_layouts/<prefix>_<raw_name>/detail/`

The combined export is produced **once per dataset run**, not once per sample. Confirmation steps:

```
# After running a dataset, list recent files under histone_layouts
find "<raw_path>/histone_layouts" -type f -maxdepth 2 -printf "%TY-%Tm-%Td %TH:%TM %p\n" | sort | tail
-n 30

# Search specifically for the ratios export
find "<raw_path>/histone_layouts" -type f -iname "*histone*ratio*.xls*" -o -iname "*histone*ratio*.tsv*"
-o -iname "*ratios*.xls*" | sort
```

If multiple candidates appear, pick the one that is:

- updated at the end of the run, and
- has *all samples* in columns (not just one `01_<raw_name>` sample).

Cohort export contract (documented from the writer)

Once you have identified the writer function, document the export as a **contract**. The contract has three layers:

1. File discovery (where it is written)
2. File format (how to parse it safely)
3. Semantic invariants (what the numbers mean)

File discovery (what the pipeline guarantees)

Guaranteed by the pipeline:

- `histone_layouts/` is created under `raw_path` during a run.
- each run creates per-sample folders:
`histone_layouts/<prefix>_<raw_name>/` and
`histone_layouts/<prefix>_<raw_name>/detail/`

Not guaranteed until you confirm in the writer:

- exact filename of the combined cohort export
- exact directory level where it is written

Therefore, the contract should state discovery like this:

- The cohort export is written somewhere under `<raw_path>/histone_layouts/`.
- The exact filename pattern is defined in the MATLAB writer `<FUNCTION>(...)`.

Then specify the *verified* path once you read the code.

57.2 File type: “.xls” is often TSV (text)

In this project, do not trust the extension. The contract must say:

- Treat the file as **tab-separated text** unless you have verified it is a real Excel binary.
- Use a text reader first (R: `readr::read_tsv`, Python: `read_csv(sep="\t")`).

57.3 Header schema (two-row header)

Most EpiProfile-style cohort exports use a **2-row header**:

- Row 1: sample names (often repeated across metrics)
- Row 2: metric labels (e.g., `Ratio`, `Area`, `RT(min)`)

Column 1 is the feature identifier (`Peptide`, `Feature`, or similar).

The contract must define:

- how many header rows (usually 2)
- what column 1 is called
- how sample/metric columns are constructed

Preferred contract wording (parsing-oriented):

- The file has exactly 2 header rows. Data begins on row 3.
- Column 1 is the feature key. Columns 2..N are sample-metric columns.
- Sample-metric columns are parsed by pairing `row1[j]` (Sample) and `row2[j]` (Metric).

57.4 Body schema (rows)

One row corresponds to one quantified feature (usually a peptideform under a peptide region).

Contract must define:

- the canonical *feature* key string as written (exactly as in column 1)
- what constitutes a “peptide group” for ratio normalization (how to group rows that sum to ~1)

Common invariant (must be validated on your data):

- For each peptide group and sample, the sum of **Ratio** across all peptideforms in that group is ~1 (subject to missing forms).

57.5 Metrics (columns)

Document the metric set exactly as written by the MATLAB writer. Typical metrics include:

- **Ratio** (relative abundance)
- **Area** (AUC)
- **RT(min)** (representative retention time)

The contract must define:

- allowed metric labels (exact strings)
- expected numeric type
- missing value representation (blank / **NaN** / **0**)

57.6 Minimal parsing template (Python)

This is the conservative parser pattern that matches a 2-row header TSV:

```
import pandas as pd

path = "histone_ratios.xls" # treat as TSV text

raw = pd.read_csv(path, sep="\t", header=None, dtype=str)

h1 = raw.iloc[0].tolist() # samples
h2 = raw.iloc[1].tolist() # metrics

dat = raw.iloc[2:].copy()
dat.columns = ["Feature"] + [f"{h1[j]}__{h2[j]}" for j in range(1, raw.shape[1])]

long = dat.melt(id_vars=["Feature"], var_name="SampleMetric", value_name="Value")
long[["Sample", "Metric"]] = long["SampleMetric"].str.split("__", n=1, expand=True)

long["Value"] = pd.to_numeric(long["Value"], errors="coerce")
long = long.drop(columns=["SampleMetric"])
```

Contract note:

- This parser assumes **exactly** 2 header rows.
- If your export deviates, fix header handling here, not downstream analysis.

58. Export validation (must be part of the contract)

The contract is not complete without validation checks you can run after every export.

58.1 Structural checks

- File exists.
- It is parseable as TSV with at least 3 rows.
- Header rows have the same number of columns.
- Column 1 exists and is non-empty for all data rows.
- Metrics include at least `Ratio` (and optionally `Area`, `RT(min)`).

58.2 Ratio invariants

For each sample and peptide group:

- `Ratio >= 0`
- `sum(Ratio)` across peptideforms is approximately 1 (allow a tolerance; missing forms and filtering can break exact equality)

58.3 Quick validator skeleton (Python)

You will fill in the peptide-group parsing rule once the writer reveals how features are encoded.

```
import pandas as pd

def read_two_header_tsv(path: str) -> pd.DataFrame:
    raw = pd.read_csv(path, sep="\t", header=None, dtype=str)
    h1 = raw.iloc[0].tolist()
    h2 = raw.iloc[1].tolist()
    dat = raw.iloc[2:].copy()
    dat.columns = ["Feature"] + [f"{h1[j]}__{h2[j]}" for j in range(1, raw.shape[1])]
    long = dat.melt(id_vars=["Feature"], var_name="SampleMetric", value_name="Value")
    long[["Sample", "Metric"]] = long["SampleMetric"].str.split("__", n=1, expand=True)
    long["Value"] = pd.to_numeric(long["Value"], errors="coerce")
    return long.drop(columns=["SampleMetric"])

def validate_ratios(long: pd.DataFrame) -> pd.DataFrame:
    # TODO: implement parse_peptide_group(feature) based on writer output
    # long["PeptideGroup"] = long["Feature"].apply(parse_peptide_group)

    # Placeholder: treat Feature itself as group (replace this!)
    long["PeptideGroup"] = long["Feature"]

    ratios = long[long["Metric"].str.lower().eq("ratio")].copy()

    neg = ratios[ratios["Value"].notna() & (ratios["Value"] < 0)]
    if len(neg) > 0:
        raise ValueError(f"Found negative ratios: {len(neg)} rows")

    sums = (ratios
            .groupby(["Sample", "PeptideGroup"], dropna=False)["Value"]
            .sum(min_count=1)
            .reset_index(name="RatioSum"))

    return sums

# usage:
# long = read_two_header_tsv("histone_ratios.xls")
# sums = validate_ratios(long)
# print(sums.sort_values("RatioSum").head(20))
```

59. What you must extract from the MATLAB writer (for line-by-line documentation)

When you open the writer function, document these blocks explicitly:

1. Inputs: what arguments define sample order, output folder, and which peptides/features are included.
2. Output path construction: how the filename and directory are built.
3. Header writing:
 - how row 1 sample names are constructed (raw_name? prefix? cleaned?)
 - how row 2 metric labels are set and in what order
4. Row generation:
 - the source arrays/matrices for Ratio/Area/RT
 - the exact string used for **Feature** (and how it encodes peptide/region/mod)
5. Formatting:
 - delimiter (**\t** vs comma)
 - numeric formatting (**%g**, **%.6f**, scientific notation, etc.)
6. Missing values:
 - how missing is represented in the file (empty string, **0**, **NaN**)
7. Close/flush: **fclose**, error handling, and any final logging